Apache Kafka工作负载与NetApp NFS存储

NetApp Solutions

NetApp
March 19, 2024

This PDF was generated from https://docs.netapp.com/zh-cn/netapp-solutions/data-analytics/kafka-nfs-introduction.html on March 19, 2024. Always check docs.netapp.com for the latest.

目录

Apache Kafka工作负载与NetApp NFS存储 . . . 1
TR-4947：Apache Kafka工作负载与NetApp NFS存储—功能验证和性能 . . . 1
NetApp解决方案 for fly将问题描述 for NFS重命名为Kafka工作负载 . . . 2
功能验证—错误的重命名修复 . . . 2
为什么选择适用于Kafka工作负载的NetApp NFS? . . . 7
AWS中的性能概述和验证 . . . 18
AWS FSx for NetApp ONTAP中的性能概述和验证 . . . 31
使用AFF A900内部部署进行性能概述和验证 . . . 39
结论 . . . 44
从何处查找追加信息 . . . 45

Apache Kafka工作负载与NetApp NFS存储

TR-4947：Apache Kafka工作负载与NetApp NFS存储—功能验证和性能

NetApp公司Shantanu Chakole、Karthikeyan Nagalingam和Joe Scott

Kafka是一个分布式发布订阅消息传送系统、具有一个强大的队列、可以接受大量消息数据。借助Kafka、应用程序可以非常快速地对主题进行数据写入和读取。由于Kafka具有容错能力和可扩展性、因此通常在大数据空间中使用它作为快速载入和移动多个数据流的可靠方式。使用情形包括流处理、网站活动跟踪、指标收集和监控、日志聚合、实时分析等。

虽然NFS上的正常Kafka操作运行良好、但 "重命名操作不明智" 在调整NFS上运行的Kafka集群的大小或重新分区期间、问题描述 会使应用程序崩溃。这是一个重要的问题描述 、因为出于负载平衡或维护目的、必须调整Kafka集群的大小或对其进行重新分区。您可以找到其他详细信息 "此处"。

本文档介绍了以下主题：

- 错误重命名问题和解决方案 验证
- 降低CPU利用率以缩短I/O等待时间
- Kafka代理恢复时间更快
- 云端和内部环境中的性能

为什么要对Kafka工作负载使用NFS存储？

生产应用程序中的Kafka工作负载可以在应用程序之间流式传输大量数据。此数据会保留并存储在Kafka集群中的Kafka代理节点中。Kafka也以可用性和并行性而闻名、它通过将主题划分为分区、然后在整个集群中复制这些分区来实现这一点。这最终意味着、流经Kafka集群的大量数据通常会成倍增加。NFS可以随着代理数量的变化轻松快速地重新平衡数据。对于大型环境、在代理数量发生变化时跨DAS重新平衡数据非常耗时、在大多数Kafka环境中、代理数量经常发生变化。

其他优势包括：

- 成熟度。 NFS是一种成熟的协议、这意味着实施、保护和使用NFS的大部分方面都已被充分理解。
- 开放式 NFS是一种开放式协议、其持续开发已作为一种免费的开放式网络协议记录在互联网规格中。
- 经济高效。 NFS是一种用于网络文件共享的低成本解决方案 、由于它使用现有网络基础架构、因此易于设置。
- *集中管理。*集中管理NFS可减少单个用户系统上对添加软件和磁盘空间的需求。
- 分布式。 NFS可用作分布式文件系统、从而减少了对可移动介质存储设备的需求。

为什么选择NetApp来处理Kafka工作负载？

NetApp NFS实施被视为该协议的黄金标准、用于无数企业级NAS环境。除了NetApp的信誉之外、它还具有以下优势：

- 可靠性和效率
- 可扩展性和性能
- 高可用性(NetApp ONTAP 集群中的HA配对节点)
- 数据保护
  - *灾难恢复(NetApp SnapMirror)。*您的站点发生故障、或者您希望从其他站点跳转并从您离开的位置继续。
  - 存储系统的易管理性(使用NetApp OnCommand 进行管理和管理)。
  - *负载平衡。*集群允许您从不同节点上托管的数据LIF访问不同的卷。
  - 无中断操作。 LIF或卷移动对NFS客户端是透明的。

NetApp解决方案 for fly将问题描述 for NFS重命名为Kafka工作负载

Kafka的构建假定底层文件系统符合POSIX标准：例如XFS或ext4。Kafka资源重新平衡会在应用程序仍在使用文件时删除这些文件。符合POSIX的文件系统允许取消链接以继续。但是、它仅会在对文件的所有引用均消失后删除该文件。如果底层文件系统已通过网络连接、则NFS客户端将截获取消链接调用并管理工作流。由于正在取消链接的文件存在待定打开状态、因此NFS客户端会向NFS服务器发送重命名请求、并在未链接的文件最后一次关闭时对已重命名的文件执行删除操作。此行为通常称为NFS愚蠢的重命名、它由NFS客户端进行编排。

由于此行为、使用NFSv3服务器中存储的任何Kafka代理都会遇到问题。但是、NFSv4.x协议具有一些功能、可以通过允许服务器对打开的未链接文件负责来解决此问题描述。支持此可选功能的NFS服务器会在文件打开时将所有权功能传递给NFS客户端。然后、当打开并等待处理时、NFS客户端将停止取消链接管理、并允许服务器管理此流。虽然NFSv4规范提供了实施准则、但到目前为止、还没有任何已知的NFS服务器实施支持此可选功能。

NFS服务器和NFS客户端需要进行以下更改、才能处理愚蠢的重命名问题描述：

- *对NFS客户端(Linux)所做的更改。*打开文件时、NFS服务器会通过一个标志进行响应、指示能够处理已打开文件的解除链接。NFS客户端更改允许NFS服务器在存在标志的情况下处理解除链接。NetApp已使用这些更改更新了开源Linux NFS客户端。更新后的NFS客户端现在在RHEL8.7和RHEL9.1中普遍可用。
- 对**NFS**服务器所做的更改。 NFS服务器会跟踪打开情况。现在、服务器会管理现有打开文件的取消链接、以匹配POSIX语义。关闭上次打开的文件后、NFS服务器会启动文件的实际删除、从而避免了愚蠢的重命名过程。ONTAP NFS服务器已在其最新版本ONTAP 9.12.1.中实施此功能。

通过对NFS客户端和服务器进行上述更改、Kafka可以安全地获得网络连接NFS存储的所有优势。

功能验证—错误的重命名修复

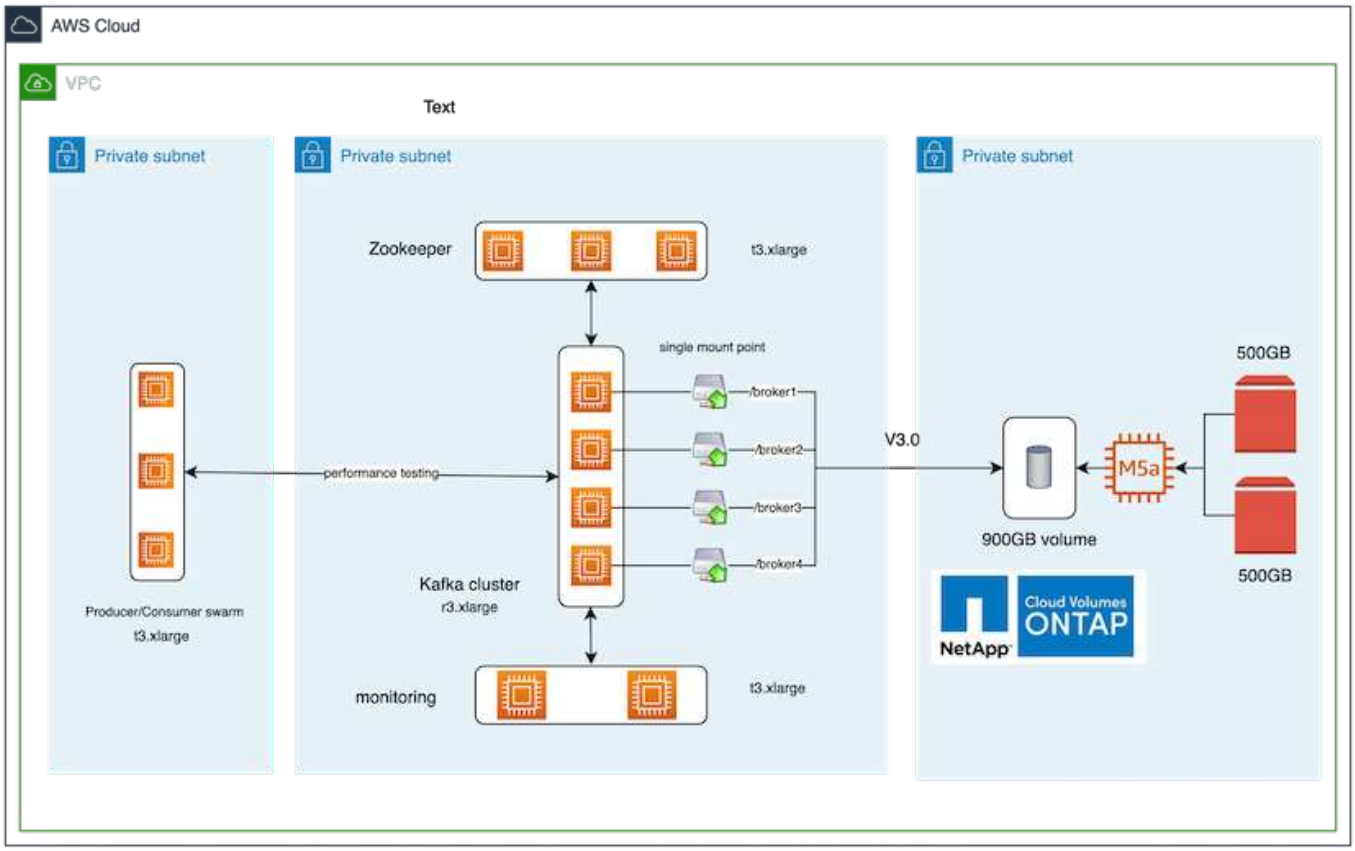
在功能验证方面、我们显示了具有NFSv3存储挂载的Kafka集群无法执行分区重新分配等Kafka操作、而具有修复程序的NFSv4上挂载的另一个集群可以执行相同的操作而不会造成任何中断。

验证设置

此设置将在AWS上运行。下表显示了用于验证的不同平台组件和环境配置。

平台组件	环境配置
Confuent Platform 7.2.1版	• 3个Zookepers—T3.xlarge • 4个代理服务器—r3.xlarge • 1个Grafana—T3.xlarge • 1个控制中心—T3.xlarge • 3个生产者/使用者
所有节点上的操作系统	RHEL8.7或更高版本
NetApp Cloud Volumes ONTAP 实例	单节点实例—M5.2xLarge

下图显示了此解决方案 的架构配置。



架构流程

- *计算。*我们使用了一个四节点Kafka集群、其中三节点Zookeeper集合在专用服务器上运行。
- *监控。*我们将两个节点用于Prometheus-Grafana组合。
- *工作负载。*为了生成工作负载、我们使用了一个单独的三节点集群、该集群可以生成并使用此Kafka集群。

- *存储。*我们使用了一个单节点NetApp Cloud Volumes ONTAP 实例、该实例连接了两个500 GB GP2 AWS-EBS卷。然后、这些卷会通过LIF作为一个NFSv4.1卷公开到Kafka集群中。

已为所有服务器选择Kafka的默认属性。Zookeeper Swarm也是如此。

测试方法

1. 更新 `-is-preserve-unlink-enabled true` 到Kafka卷、如下所示：

```
aws-shantanclastrecall-aws::*> volume create -vserver kafka_svm -volume
kafka_fg_vol01 -aggregate kafka_aggr -size 3500GB -state online -policy
kafka_policy -security-style unix -unix-permissions 0777 -junction-path
/kafka_fg_vol01 -type RW -is-preserve-unlink-enabled true
[Job 32] Job succeeded: Successful
```

2. 创建了两个类似的Kafka集群、但差别如下：
   - *集群1.*运行生产就绪型ONTAP 9.12.1的后端NFS v4.1服务器由NetApp CVO实例托管。这些代理上安装了RHEL 8.7/RHEL 9.1。
   - *集群2.*后端NFS服务器是手动创建的通用Linux NFSv3服务器。
3. 在这两个Kafka集群上都创建了一个演示主题。

   集群1：

```
[root@ip-172-30-0-160 demo]# kafka-topics --bootstrap-server=172.30.0.160:9092,172.30.0.172:9092,172.30.0.188:90
92,172.30.0.123:9092 --describe --topic __a_demo_topic
Topic: __a_demo_topic   TopicId: 2ty29xfhQLq65HKsUQv-pg PartitionCount: 4       ReplicationFactor: 2    Configs:
 min.insync.replicas=1,segment.bytes=1073741824
        Topic: __a_demo_topic   Partition: 0    Leader: 4       Replicas: 4,1   Isr: 4,1        Offline:
        Topic: __a_demo_topic   Partition: 1    Leader: 2       Replicas: 2,4   Isr: 2,4        Offline:
        Topic: __a_demo_topic   Partition: 2    Leader: 3       Replicas: 3,2   Isr: 3,2        Offline:
        Topic: __a_demo_topic   Partition: 3    Leader: 1       Replicas: 1,3   Isr: 1,3        Offline:
```

   集群2：

```
[root@ip-172-30-0-198 demo]# kafka-topics --bootstrap-server=172.30.0.198:9092,172.30.0.163:9092,172.30.0.221:90
92,172.30.0.204:9092 --describe --topic __a_demo_topic
Topic: __a_demo_topic   TopicId: AwQpsZTQShyeMIhaquCG3Q PartitionCount: 4       ReplicationFactor: 2    Configs:
 min.insync.replicas=1,segment.bytes=1073741824
        Topic: __a_demo_topic   Partition: 0    Leader: 2       Replicas: 2,3   Isr: 2,3        Offline:
        Topic: __a_demo_topic   Partition: 1    Leader: 3       Replicas: 3,1   Isr: 3,1        Offline:
        Topic: __a_demo_topic   Partition: 2    Leader: 1       Replicas: 1,4   Isr: 1,4        Offline:
        Topic: __a_demo_topic   Partition: 3    Leader: 4       Replicas: 4,2   Isr: 4,2        Offline:
```

4. 数据已加载到这两个集群的新创建主题中。这是使用默认Kafka软件包中提供的producer-perf-test工具包实现的：

```
./kafka-producer-perf-test.sh --topic __a_demo_topic --throughput -1
--num-records 3000000 --record-size 1024 --producer-props acks=all
bootstrap.servers=172.30.0.160:9092,172.30.0.172:9092,172.30.0.188:9092,
172.30.0.123:9092
```

5. 已使用telnet对每个集群的Broker-1执行运行状况检查：
   - Telnet `172.30.0.160 9092`
   - Telnet `172.30.0.198 9092`

     下一个屏幕截图显示了两个集群上的代理的成功运行状况检查：

```
shantanu@shantanc-mac-0 ~ % telnet 172.30.0.160 9092
Trying 172.30.0.160...
Connected to 172.30.0.160.
Escape character is '^]'.
^[

Connection closed by foreign host.
shantanu@shantanc-mac-0 ~ % telnet 172.30.0.198 9092
Trying 172.30.0.198...
Connected to 172.30.0.198.
Escape character is '^]'.
^[
```

6. 为了触发导致使用NFSv3存储卷的Kafka集群崩溃的故障情况、我们在这两个集群上启动了分区重新分配过程。分区重新分配是使用执行的 `kafka-reassign-partitions.sh`。详细过程如下：
   a. 为了为Kafka集群中的某个主题重新分配分区、我们生成了建议的重新分配配置JSON (这是为这两个集群执行的)。

```
kafka-reassign-partitions --bootstrap
-server=172.30.0.160:9092,172.30.0.172:9092,172.30.0.188:9092,172.30.
0.123:9092 --broker-list "1,2,3,4" --topics-to-move-json-file
/tmp/topics.json --generate
```

   b. 生成的重新分配JSON随后保存在中 `/tmp/reassignment- file.json`。
   c. 实际分区重新分配过程由以下命令触发：

```
kafka-reassign-partitions --bootstrap
-server=172.30.0.198:9092,172.30.0.163:9092,172.30.0.221:9092,172.30.
0.204:9092 --reassignment-json-file /tmp/reassignment-file.json
-execute
```

7. 在完成重新分配几分钟后、对代理进行的另一项运行状况检查显示、使用NFSv3存储卷的集群运行到一个错误的重命名问题描述 中并发生崩溃、而使用NetApp ONTAP NFSv4.1存储卷的集群1则在修复后继续运行、而不会造成任何中断。

```
shantanu@shantanc-mac-0 ~ % telnet 172.30.0.160 9092
Trying 172.30.0.160...
Connected to 172.30.0.160.
Escape character is '^]'.
^[

Connection closed by foreign host.
shantanu@shantanc-mac-0 ~ % telnet 172.30.0.198 9092
Trying 172.30.0.198...
telnet: connect to address 172.30.0.198: Connection refused
telnet: Unable to connect to remote host
```

- cluster1-Broker-1处于活动状态。
- CLUSTER2-Broker-1已失效。

8. 检查Kafka日志目录后、可以明显看出、使用NetApp ONTAP NFSv4.1存储卷并进行修复的集群1分配了干净的分区、而使用通用NFSv3存储的集群2则不是由于错误的重命名问题而导致崩溃。下图显示了集群2的分区重新平衡、这会导致在NFSv3存储上对问题描述 进行重命名、操作很不明智。

```
/demo/broker_demo_1/__a_demo_topic-1.b31a8dd60fd443b283ffda2ecca9c2b9-delete:
total 40
drwxr-xr-x.   2 nobody nobody  4096 Sep 19 10:37 .
drwxr-xr-x. 246 nobody nobody 32768 Sep 19 10:36 ..
-rw-r--r--.   1 nobody nobody     5 Sep 19 10:22 .nfs0000000025f9008400000045
-rw-r--r--.   1 nobody nobody     0 Sep 19 10:25 .nfs0000000025f91d6800000048

/demo/broker_demo_1/__a_demo_topic-2:
total 832592
drwxr-xr-x.   2 nobody nobody      4096 Sep 19 10:26 .
drwxr-xr-x. 246 nobody nobody     32768 Sep 19 10:36 ..
-rw-r--r--.   1 nobody nobody         5 Sep 19 10:22 .nfs0000000025f91d5500000046
-rw-r--r--.   1 nobody nobody         0 Sep 19 10:25 .nfs0000000025f91fce00000047
-rw-r--r--.   1 nobody nobody  10485760 Sep 19 10:24 00000000000000000000.index
-rw-r--r--.   1 nobody nobody 848113134 Sep 19 10:24 00000000000000000000.log
-rw-r--r--.   1 nobody nobody  10485756 Sep 19 10:24 00000000000000000000.timeindex
-rw-r--r--.   1 nobody nobody         0 Sep 19 10:16 leader-epoch-checkpoint
-rw-r--r--.   1 nobody nobody        43 Sep 19 10:16 partition.metadata
```

下图显示了使用NetApp NFSv4.1存储重新平衡集群1的全新分区。

```
/demo/broker_demo_1/__a_demo_topic-0:
total 710932
drwxr-xr-x.  2 nobody nobody      4096 Sep 19 10:26 .
drwxr-xr-x. 85 nobody nobody      8192 Sep 19 10:37 ..
-rw-r--r--.  1 nobody nobody  10485760 Sep 19 10:25 00000000000000000000.index
-rw-r--r--.  1 nobody nobody 724167522 Sep 19 10:25 00000000000000000000.log
-rw-r--r--.  1 nobody nobody  10485756 Sep 19 10:25 00000000000000000000.timeindex
-rw-r--r--.  1 nobody nobody         0 Sep 19 10:15 leader-epoch-checkpoint
-rw-r--r--.  1 nobody nobody        43 Sep 19 10:15 partition.metadata

/demo/broker_demo_1/__a_demo_topic-2:
total 780016
drwxr-xr-x.  2 nobody nobody      4096 Sep 19 10:35 .
drwxr-xr-x. 85 nobody nobody      8192 Sep 19 10:37 ..
-rw-r--r--.  1 nobody nobody  10485760 Sep 19 10:36 00000000000000000000.index
-rw-r--r--.  1 nobody nobody 794575786 Sep 19 10:36 00000000000000000000.log
-rw-r--r--.  1 nobody nobody  10485756 Sep 19 10:36 00000000000000000000.timeindex
-rw-r--r--.  1 nobody nobody         0 Sep 19 10:35 leader-epoch-checkpoint
-rw-r--r--.  1 nobody nobody        43 Sep 19 10:35 partition.metadata
```

为什么选择适用于**Kafka**工作负载的**NetApp NFS**？

现在、在使用Kafka的NFS存储中、有一个解决方案 用于愚蠢地重命名问题描述 、您可以创建强大的部署、利用NetApp ONTAP 存储来处理Kafka工作负载。这样不仅可以显著降低运营开销、还可以为Kafka集群带来以下优势：

- *降低Kafka代理的CPU利用率。*使用分解的NetApp ONTAP 存储可将磁盘I/O操作与代理分离、从而减少其CPU占用空间。
- *代理恢复时间更快。*由于分离式NetApp ONTAP 存储在Kafka代理节点之间共享、因此与传统Kafka部署相比、新的计算实例可以在任意时间点替换损坏的代理、而无需重建数据。
- *存储效率。*由于应用程序的存储层现在通过NetApp ONTAP 进行配置、因此客户可以利用ONTAP 带来的存储效率的所有优势、例如实时数据压缩、重复数据删除和数据缩减。

我们在本节详细讨论的测试案例中对这些优势进行了测试和验证。

降低了**Kafka**代理的**CPU**利用率

我们发现、当我们在两个sperate Kafka集群上运行类似的工作负载时、总CPU利用率低于其DAS对应项、这两个集群的技术规格相同、但存储技术却不同。当Kafka集群使用ONTAP 存储时、不仅整体CPU利用率较低、而且CPU利用率的增加也显示出比基于DAS的Kafka集群更温和的梯度。

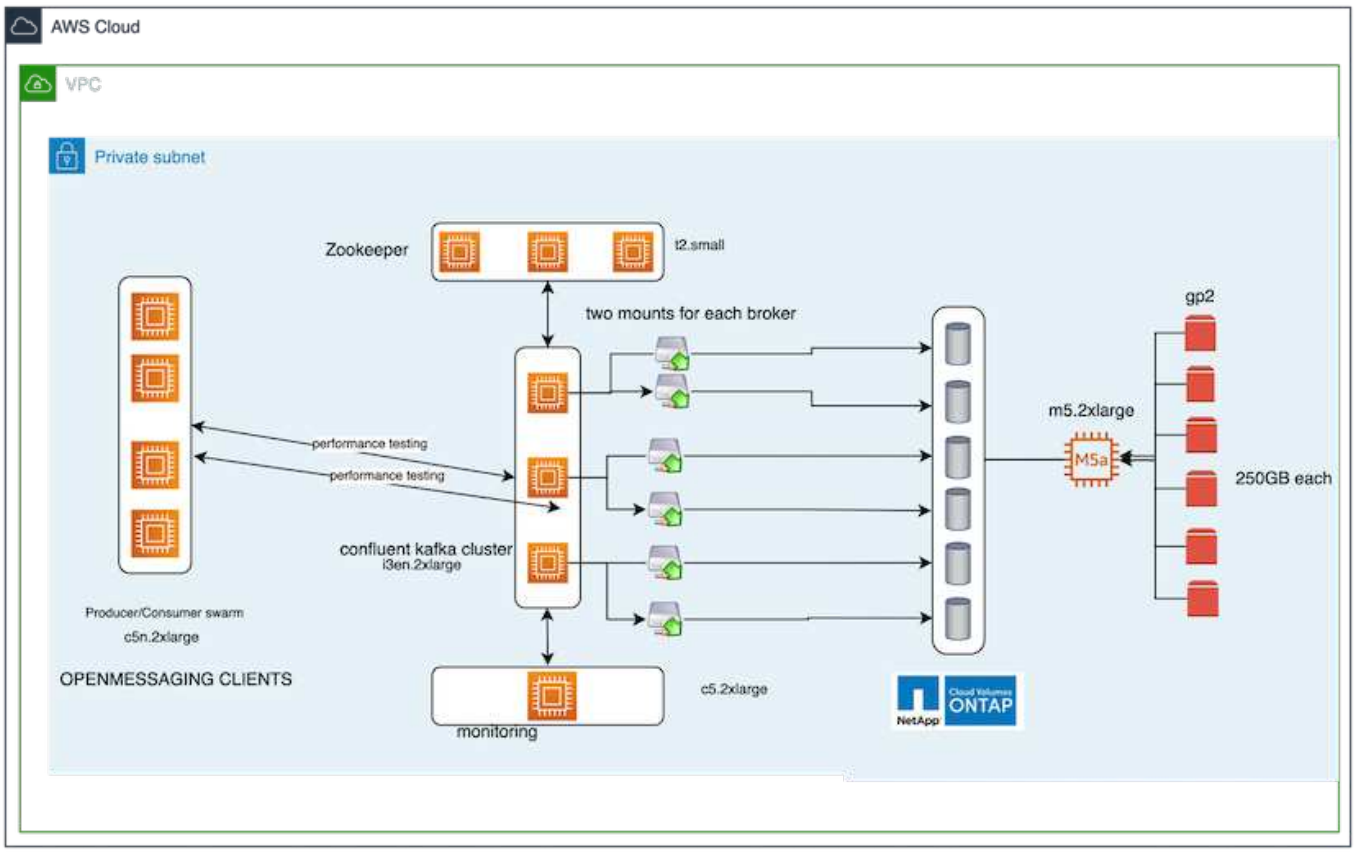
架构设置

下表显示了用于展示CPU利用率降低情况的环境配置。

平台组件	环境配置
Kafka 3.2.3基准工具：OpenMessaging	• 3个Zookepers—T2.Small • 3个代理服务器—i3en.2xlarge • 1个Grafana—c5n.2xlarge • 4个生产者/使用者—c5n.2xlarge
所有节点上的操作系统	RHEL 8.7或更高版本
NetApp Cloud Volumes ONTAP 实例	单节点实例—M5.2xLarge

基准测试工具

此测试案例中使用的基准测试工具是 "Open消息 传送" 框架。Openmessaging不受供应商限制、不受语言限制；它为金融、电子商务、物联网和大数据提供行业指导；它有助于跨异构系统和平台开发消息传送和流式传输应用程序。下图展示了Open消息 客户端与Kafka集群的交互。



- *计算。*我们使用了一个三节点Kafka集群、其中三节点Zookeeper集合在专用服务器上运行。每个代理都通过一个专用LIF将两个NFSv4.1挂载点连接到NetApp CVO实例上的一个卷。
- *监控。*我们将两个节点用于Prometheus-Grafana组合。为了生成工作负载、我们提供了一个单独的三节点集群、该集群可以生成并使用此Kafka集群。
- *存储。*我们使用了一个单节点NetApp Cloud Volumes ONTAP 实例、该实例上挂载了六个250 GB GP2 AWS-EBS卷。然后、这些卷会通过专用LIF作为六个NFSv4.1卷公开到Kafka集群中。
- *配置。*本测试用例中的两个可配置元素是Kafka代理和Open消息 工作负载。

- *代理配置*为Kafka代理选择了以下规格。我们对所有测量结果使用了复制因子3、如下所示。

```
broker.id=1
advertised.listeners=PLAINTEXT://172.30.0.185:9092
log.dirs=/mnt/data-1
zookeeper.connect=172.30.0.13:2181,172.30.0.108:2181,172.30.0.253:2181
num.replica.fetchers=8
message.max.bytes=10485760
replica.fetch.max.bytes=10485760
num.network.threads=8
default.replication.factor=3
replica.lag.time.max.ms=100000000
replica.fetch.max.bytes=1048576
replica.fetch.wait.max.ms=500
num.replica.fetchers=1
replica.high.watermark.checkpoint.interval.ms=5000
fetch.purgatory.purge.interval.requests=1000
producer.purgatory.purge.interval.requests=1000
replica.socket.timeout.ms=30000
replica.socket.receive.buffer.bytes=65536
```

- 提供了以下规格：* Open消息 基准测试(OMB)工作负载配置*。我们指定了一个目标生产者比率、并在下面重点说明了这一比率。

```
name: 4 producer / 4 consumers on 1 topic
topics: 1
partitionsPerTopic: 100
messageSize: 1024
payloadFile: "payload/payload-1Kb.data"
subscriptionsPerTopic: 1
consumerPerSubscription: 4
producersPerTopic: 4
producerRate: 40000
consumerBacklogSizeGB: 0
testDurationMinutes: 5
```

测试方法

1. 创建了两个类似的集群、每个集群都有自己的一组基准集群Swarms。
    - *集群1.*基于NFS的Kafka集群。
    - *集群2.*基于DAS的Kafka集群。
2. 使用Open消息 命令、在每个集群上触发类似的工作负载。

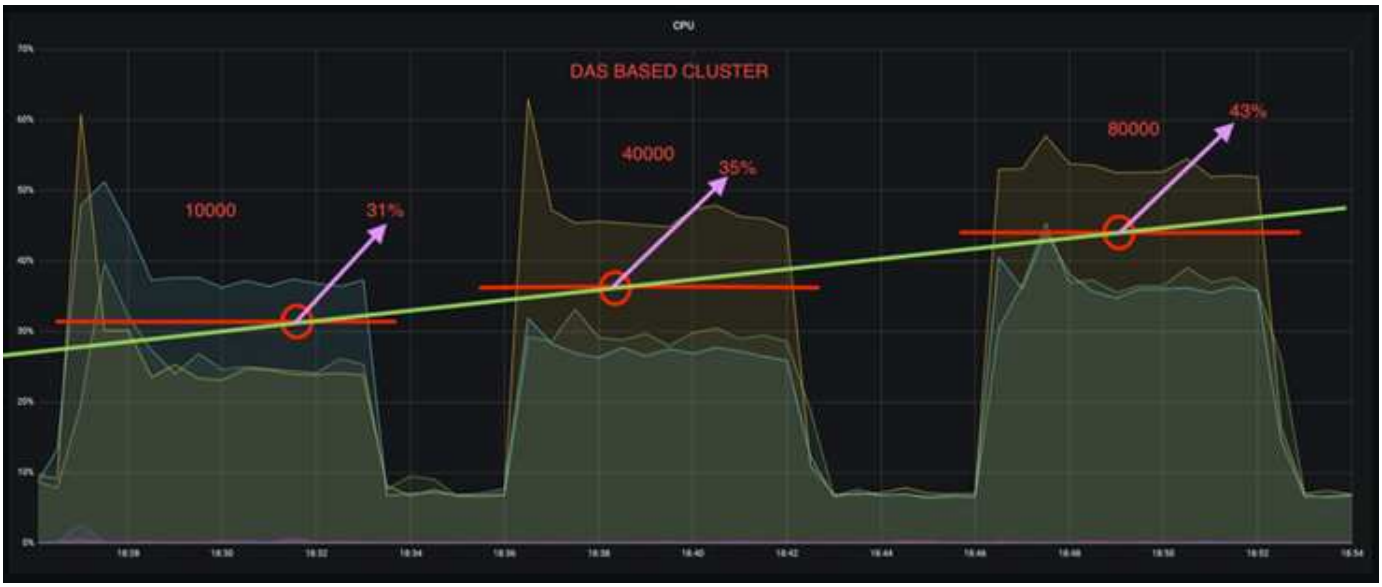
```
sudo bin/benchmark --drivers driver-kafka/kafka-group-all.yaml
workloads/1-topic-100-partitions-1kb.yaml
```

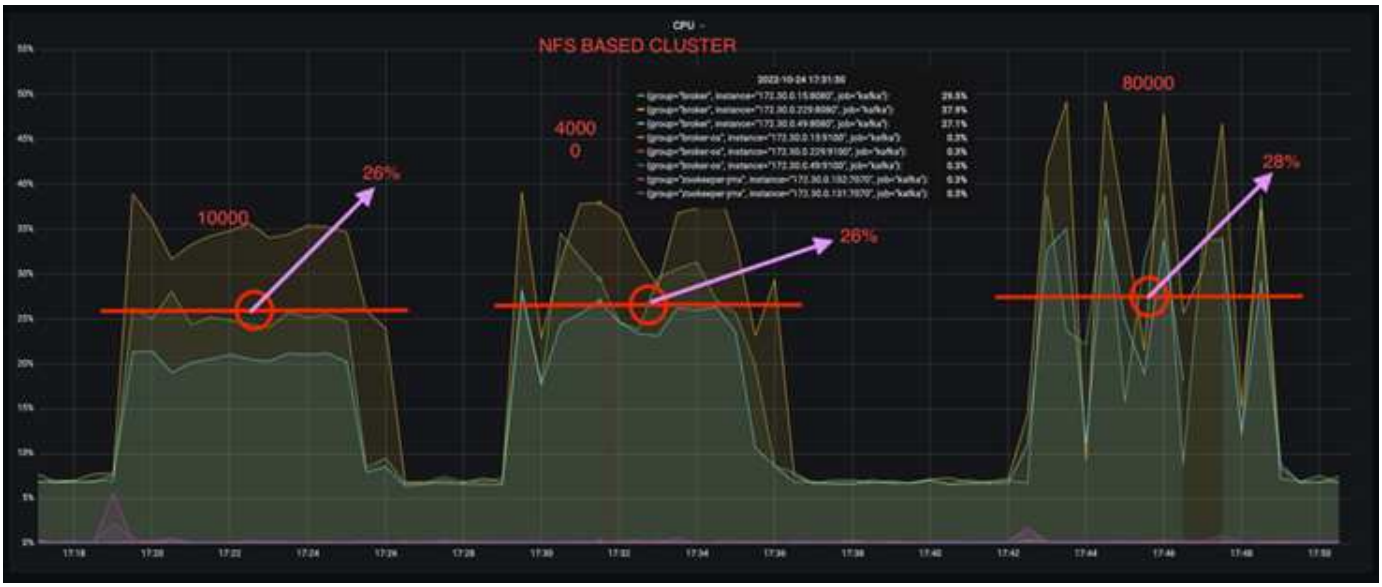
3. 生产率配置在四次迭代中增加、CPU利用率记录在Grafana中。生产率设置为以下级别：
    - 10、000
    - 40,000
    - 80、000
    - 100、000

观察结果

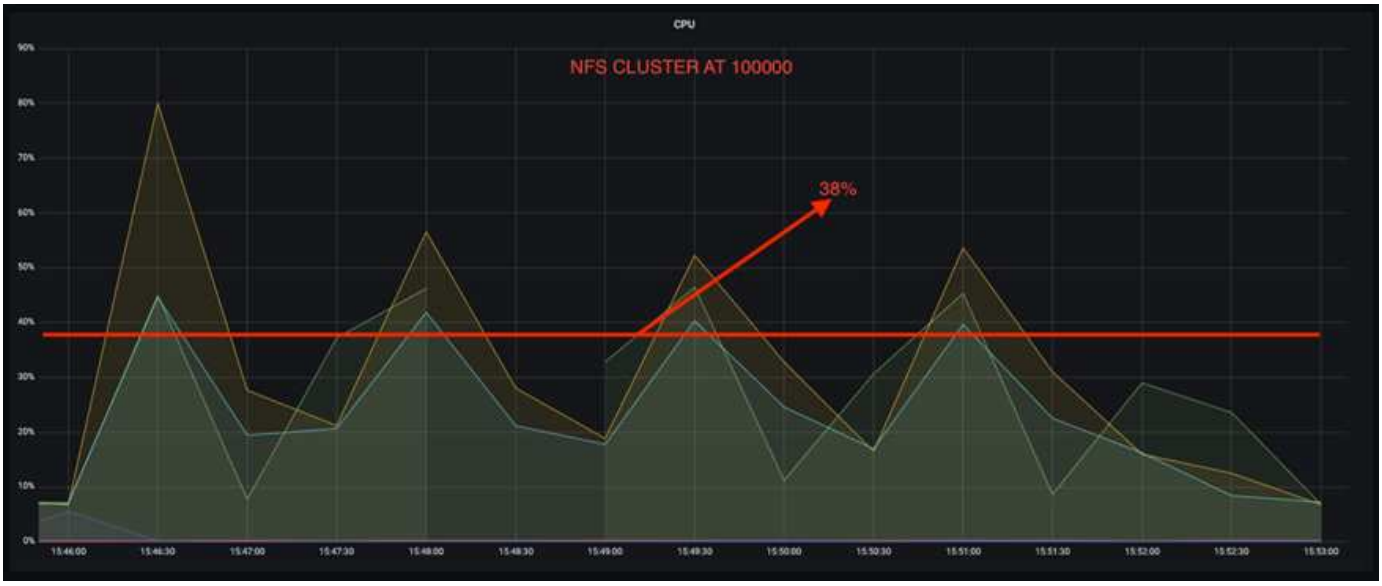
将NetApp NFS存储与Kafka结合使用具有两个主要优势：

- *您可以将CPU利用率降低近三分之一。*与DAS SSD相比、NFS在类似工作负载下的整体CPU利用率更低；节省量从较低生产率的5%到较高生产率的32%不等。
- *在较高的生产率下、CPU利用率漂移减少了三倍。*正如预期的那样、随着生产率的增加、CPU利用率的增加也出现了上升趋势。但是、使用DAS的Kafka代理的CPU利用率从较低生产率的31%上升到较高生产率的70%、即增加39%。但是、在NFS存储后端、CPU利用率从26%上升到38%、增加了12%。





此外、当消息达到100、000时、DAS显示的CPU利用率比NFS集群高。



代理恢复速度更快

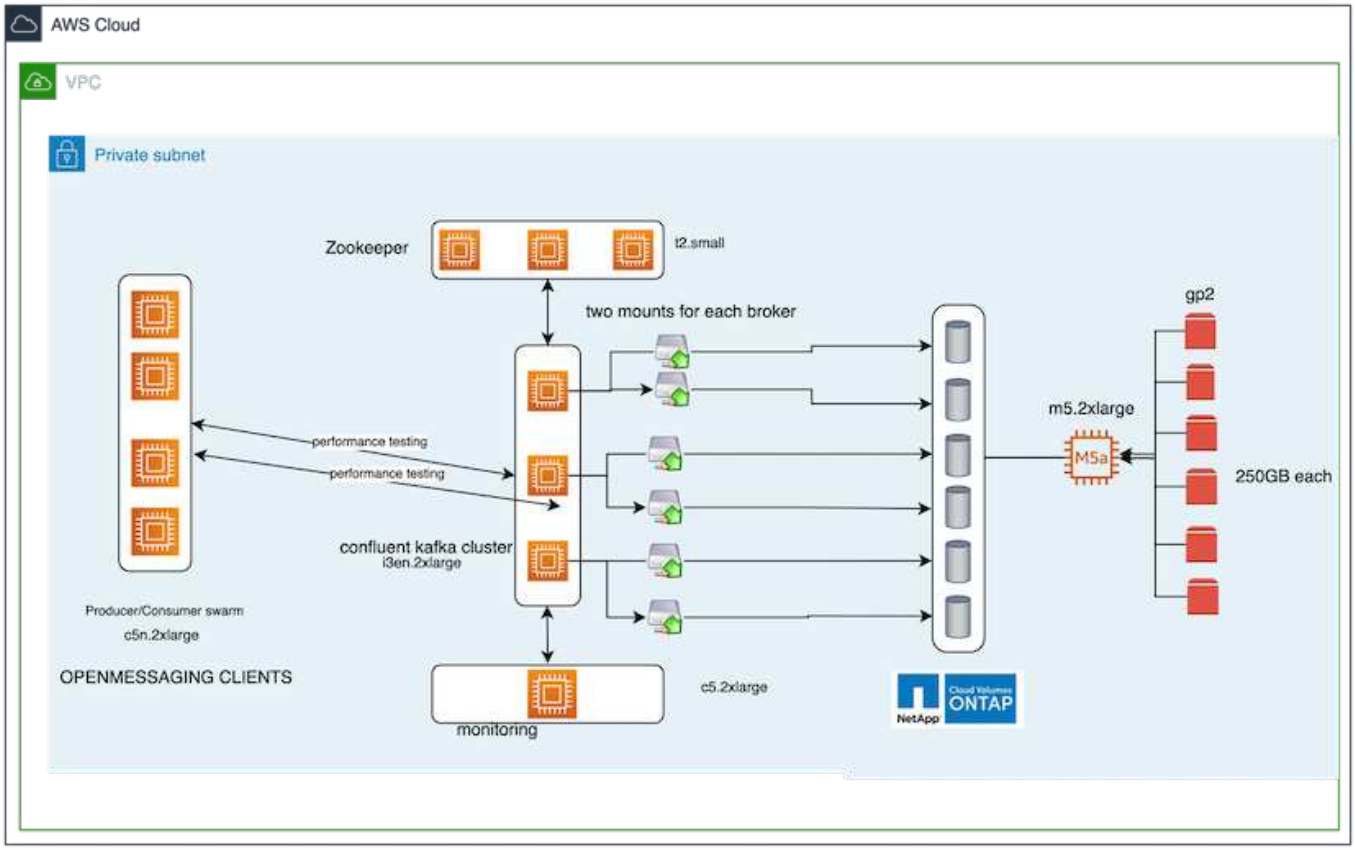
我们发现、Kafka代理在使用共享NetApp NFS存储时恢复速度更快。当Kafka集群中的代理崩溃时、可以使用具有相同代理ID的运行状况良好的代理来替换此代理。执行此测试案例后、我们发现、对于基于DAS的Kafka集群、集群会在新添加的运行状况良好的代理上重建数据、这非常耗时。对于基于NetApp NFS的Kafka集群、替代代理将继续从先前的日志目录读取数据并以更快的速度恢复。

架构设置

下表显示了使用NAS的Kafka集群的环境配置。

平台组件	环境配置
Kafka 3.2.3	• 3个Zookepers—T2.Small • 3个代理服务器—i3en.2xlarge • 1个Grafana—c5n.2xlarge • 4个生产者/使用者—c5n.2xlarge • 1个备份Kafka节点—i3en.2xlarge
所有节点上的操作系统	RHEL8.7或更高版本
NetApp Cloud Volumes ONTAP 实例	单节点实例—M5.2xLarge

下图展示了基于NAS的Kafka集群的架构。



- *计算。*一种三节点Kafka集群、其中三节点zookeeper集合在专用服务器上运行。每个代理都有两个NFS挂载点、可通过专用LIF连接到NetApp CVO实例上的一个卷。
- 监控。 Prometheus-Grafana组合的两个节点。在生成工作负载时、我们会使用一个单独的三节点集群来生成此Kafka集群并将其使用。
- *存储。*一个单节点NetApp Cloud Volumes ONTAP 实例、该实例上挂载了六个250 GB GP2 AWS-EBS卷。然后、这些卷会通过专用LIF作为六个NFS卷公开到Kafka集群中。
- *代理配置。*本测试用例中的一个可配置元素是Kafka代理。为Kafka代理选择了以下规格。。`replica.lag.time.mx.ms` 设置为高值、因为这决定了从ISR列表中删除特定节点的速度。在不良节点和运行状况良好的节点之间切换时、您不希望从ISR列表中排除该代理ID。

```
broker.id=1
advertised.listeners=PLAINTEXT://172.30.0.185:9092
log.dirs=/mnt/data-1
zookeeper.connect=172.30.0.13:2181,172.30.0.108:2181,172.30.0.253:2181
num.replica.fetchers=8
message.max.bytes=10485760
replica.fetch.max.bytes=10485760
num.network.threads=8
default.replication.factor=3
replica.lag.time.max.ms=100000000
replica.fetch.max.bytes=1048576
replica.fetch.wait.max.ms=500
num.replica.fetchers=1
replica.high.watermark.checkpoint.interval.ms=5000
fetch.purgatory.purge.interval.requests=1000
producer.purgatory.purge.interval.requests=1000
replica.socket.timeout.ms=30000
replica.socket.receive.buffer.bytes=65536
```

测试方法

1. 创建了两个类似的集群：
   - 基于EC2的融合集群。
   - 基于NetApp NFS的融合集群。
2. 创建了一个备用Kafka节点、其配置与原始Kafka集群中的节点相同。
3. 在每个集群上创建了一个示例主题、并在每个代理上填充了大约110 GB的数据。
   - 基于* EC2的集群。*已映射Kafka代理数据目录 `/mnt/data-2` (在下图中、为cluster1的Broker-1 (左端子)。
   - 基于**NetApp NFS**的集群。 Kafka代理数据目录挂载在NFS点上 `/mnt/data` (在下图中、为cluster2的Broker-1 [右端子])。

4. 在每个集群中、Broker-1都已终止、以触发失败的代理恢复过程。

5. 代理终止后、代理IP地址将作为二级IP分配给备用代理。之所以需要这样做、是因为Kafka集群中的代理可通过以下方式进行标识：
    - 通过将故障代理IP重新分配给备用代理来分配* IP地址*。
    - 代理**ID**。此ID已在备用代理中配置 `server.properties`。
6. 分配IP后、在备用代理上启动了Kafka服务。
7. 一段时间后、服务器日志被提取、用于检查在集群中的替代节点上构建数据所用的时间。

观察结果

Kafka代理恢复速度几乎是原来的九倍。我们发现、与在Kafka集群中使用DAS SSD相比、使用NetApp NFS共享存储时、恢复发生故障的代理节点所需的时间要快得多。对于1 TB的主题数据、基于DAS的集群的恢复时间为48分钟、而基于NetApp-NFS的Kafka集群的恢复时间不到5分钟。

我们发现、基于EC2的集群需要10分钟才能在新代理节点上重建110 GB的数据、而基于NFS的集群则需要3分钟才能完成恢复。我们还在日志中观察到、EC2分区的使用者偏移量为0、而在NFS集群上、使用者偏移量是从先前的代理中获取的。

```
[2022-10-31 09:39:17,747] INFO [LogLoader partition=test-topic-51R3EWs-
0000-55, dir=/mnt/kafka-data/broker2] Reloading from producer snapshot and
rebuilding producer state from offset 583999 (kafka.log.UnifiedLog$)
[2022-10-31 08:55:55,170] INFO [LogLoader partition=test-topic-qbVsEZg-
0000-8, dir=/mnt/data-1] Loading producer state till offset 0 with message
format version 2 (kafka.log.UnifiedLog$)
```

基于**DAS**的集群

1. 备份节点从08：55：53、730开始。

```
[2022-10-31 08:55:53,661] INFO Setting -D jdk.tls.rejectClientInitiatedRenegotia
[2022-10-31 08:55:53,727] INFO Registered signal handlers for TERM, INT, HUP (org
[2022-10-31 08:55:53,730] INFO starting (kafka.server.KafkaServer)
[2022-10-31 08:55:53,730] INFO Connecting to zookeeper on 172.30.0.17:2181,172.3
[2022-10-31 08:55:53,755] INFO [ZooKeeperClient Kafka server] Initializing a new
```

2. 数据重建过程于09：05：24、860结束。处理110 GB的数据大约需要10分钟。

```
[2022-10-31 09:05:24,860] INFO [ReplicaFetcherManager on broker 1] Removed fetcher for
partitions HashSet(test-topic-qbVsEZg-0000-95, test-topic-qbVsEZg-0000-5,
test-topic-qbVsEZg-0000-41, test-topic-qbVsEZg-0000-23, test-topic-qbVsEZg-0000-11,
test-topic-qbVsEZg-0000-47, test-topic-qbVsEZg-0000-83, test-topic-qbVsEZg-0000-35,
test-topic-qbVsEZg-0000-89, test-topic-qbVsEZg-0000-71, test-topic-qbVsEZg-0000-53,
test-topic-qbVsEZg-0000-29, test-topic-qbVsEZg-0000-59, test-topic-qbVsEZg-0000-77,
test-topic-qbVsEZg-0000-65, test-topic-qbVsEZg-0000-17)
(kafka.server.ReplicaFetcherManager)
```

基于**NFS**的集群

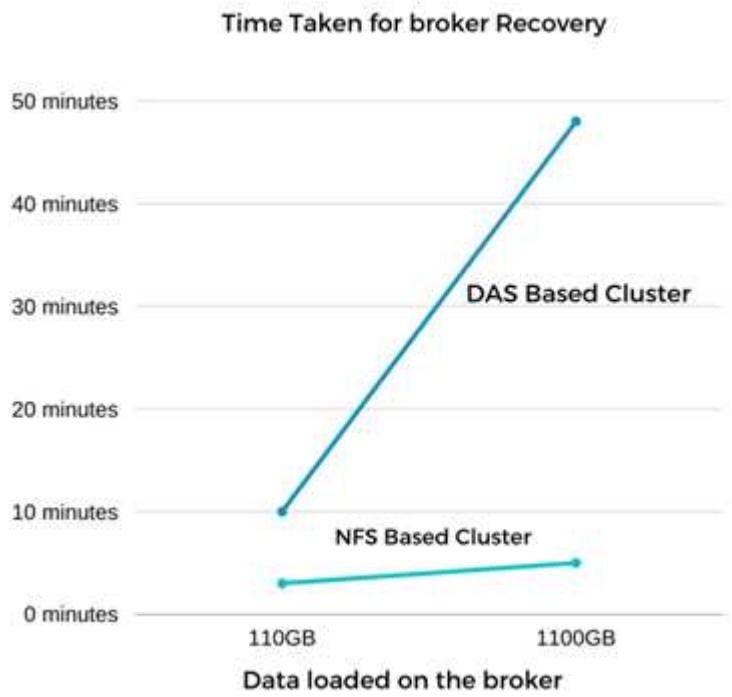
1. 备份节点的启动时间为09：39：17、213。下面突出显示了起始日志条目。

```
[2022-10-31 09:39:17,142] INFO Setting -D jdk.tls.rejectClientInitiatedRenegotiati
[2022-10-31 09:39:17,211] INFO Registered signal handlers for TERM, INT, HUP (org.
[2022-10-31 09:39:17,213] INFO starting (kafka.server.KafkaServer)
[2022-10-31 09:39:17,214] INFO Connecting to zookeeper on 172.30.0.22:2181,172.30.
[2022-10-31 09:39:17,238] INFO [ZooKeeperClient Kafka server] Initializing a new s
[2022-10-31 09:39:17,244] INFO Client environment:zookeeper.version=3.6.3--6401e4a
[2022-10-31 09:39:17,244] INFO Client environment:host.name=ip-172-30-0-110.ec2.in
[2022-10-31 09:39:17,244] INFO Client environment:java.version=11.0.17 (org.apache
```

2. 数据重建过程于09：42：29、115结束。处理110 GB的数据大约需要3分钟。

```
[2022-10-31 09:42:29,115] INFO [GroupMetadataManager brokerId=1] Finished loading offsets
and group metadata from __consumer_offsets-20 in 28478 milliseconds for epoch 3, of which
28478 milliseconds was spent in the scheduler.
(kafka.coordinator.group.GroupMetadataManager)
```

对于包含大约1 TB数据的代理、重复执行此测试、对于DAS、此测试需要大约48分钟、对于NFS、此测试需要3分钟。下图显示了这些结果。



存储效率

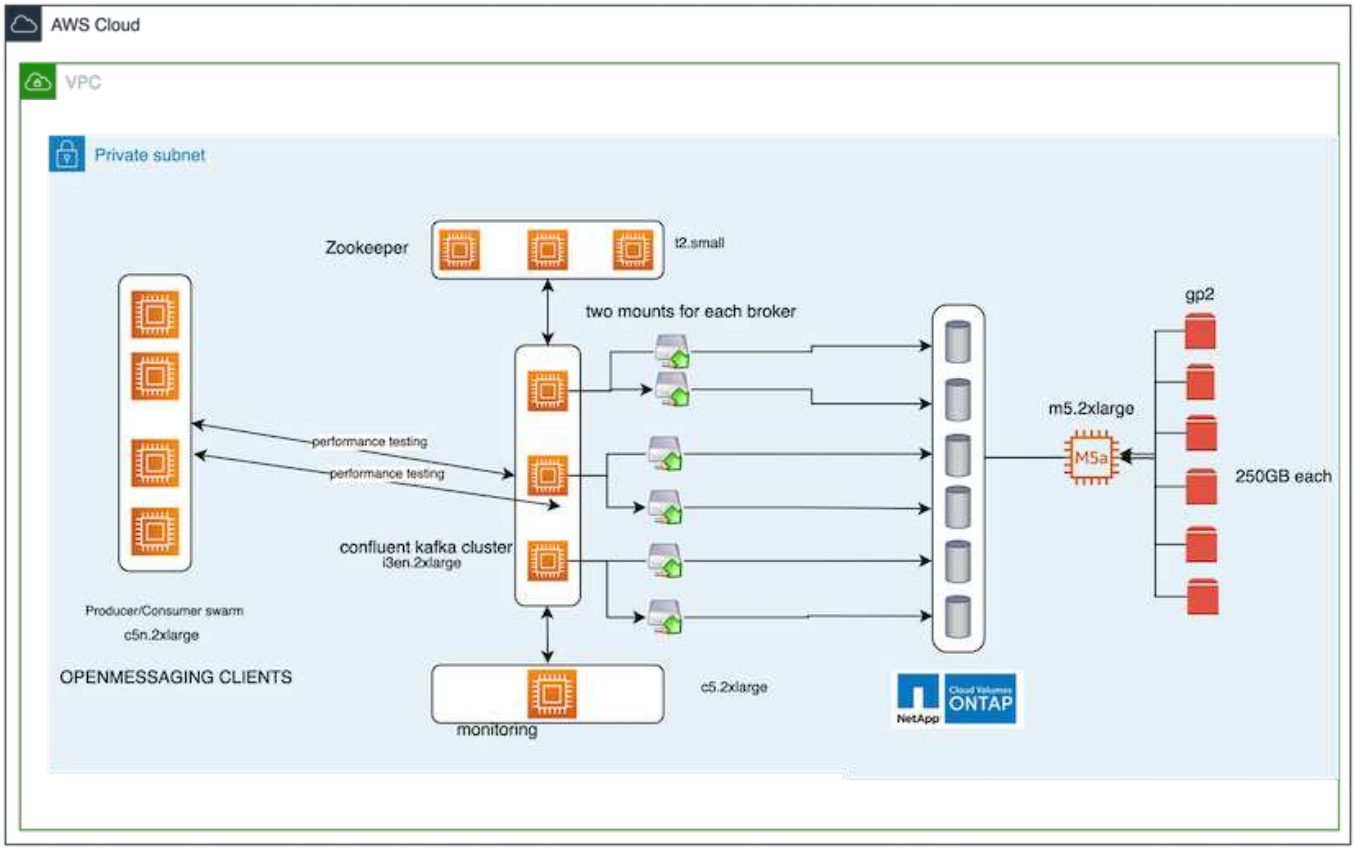
由于Kafka集群的存储层是通过NetApp ONTAP 配置的、因此我们获得了ONTAP 的所有存储效率功能。测试方法是、在Cloud Volumes ONTAP 上配置了NFS存储的Kafka集群上生成大量数据。我们可以看到、由于ONTAP功能、空间显著减少。

架构设置

下表显示了使用NAS的Kafka集群的环境配置。

平台组件	环境配置
Kafka 3.2.3	• 3个Zookepers—T2.Small • 3个代理服务器—i3en.2xlarge • 1个Grafana—c5n.2xlarge • 4个生产者/使用者—c5n.2xlarge *
所有节点上的操作系统	RHEL8.7或更高版本
NetApp Cloud Volumes ONTAP 实例	单节点实例—M5.2xLarge

下图展示了基于NAS的Kafka集群的架构。



- *计算。*我们使用了一个三节点Kafka集群、其中三节点Zookeeper集合在专用服务器上运行。每个代理都通过一个专用LIF在NetApp CVO实例上有两个NFS挂载点到一个卷。
- *监控。*我们将两个节点用于Prometheus-Grafana组合。为了生成工作负载、我们使用了一个单独的三节点集群、该集群可能会生成此Kafka集群并将其占用。
- *存储。*我们使用了一个单节点NetApp Cloud Volumes ONTAP 实例、该实例上挂载了六个250 GB GP2 AWS-EBS卷。然后、这些卷会通过专用LIF作为六个NFS卷公开到Kafka集群中。
- *配置。*此测试案例中可配置的元素是Kafka代理。

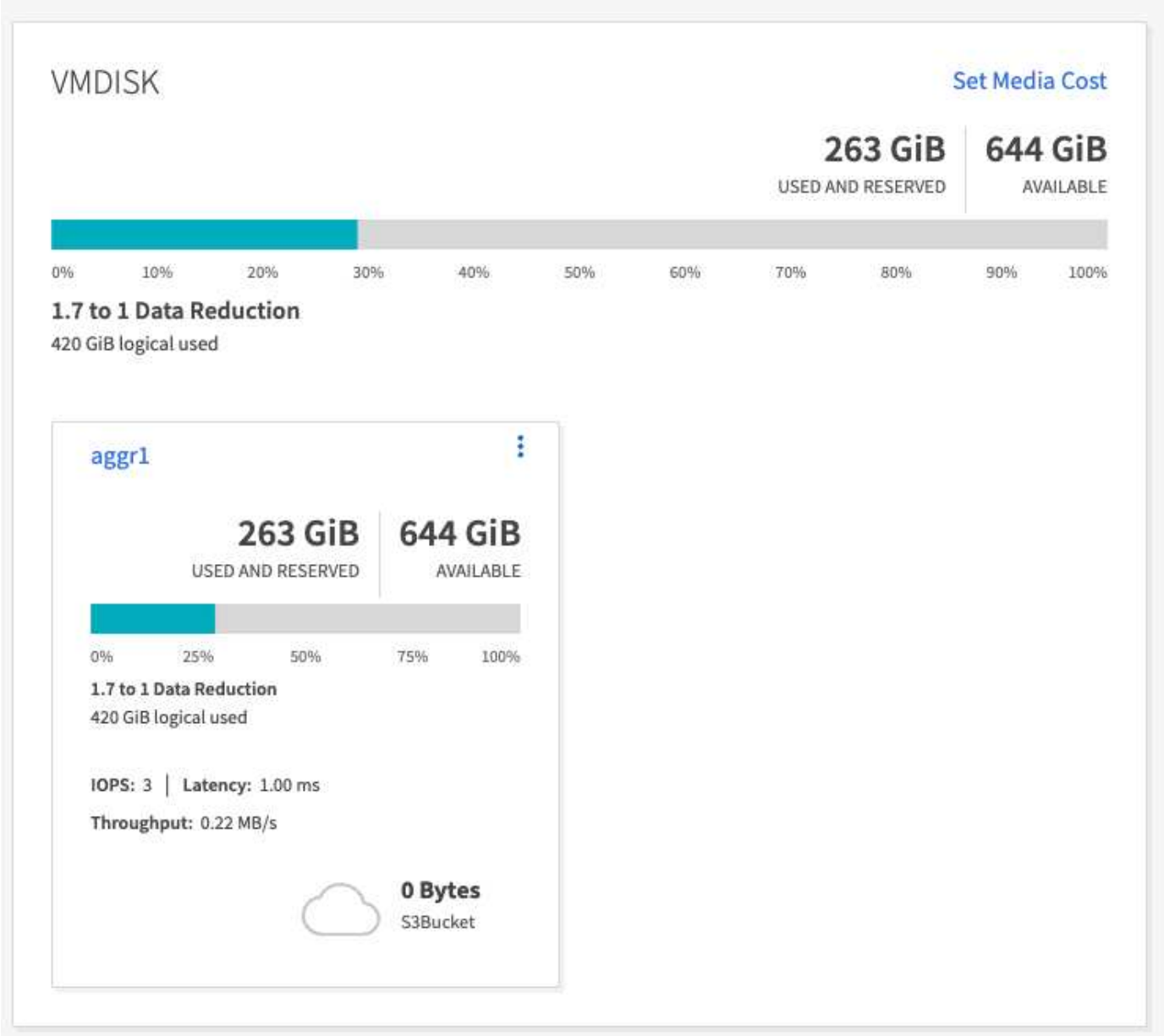
在生产商端关闭了数据压缩、从而使生产商能够生成高吞吐量。而是由计算层处理存储效率。

测试方法

1. 已按照上述规格配置Kafka集群。
2. 在集群上、使用Open消息 基准工具生成了大约350 GB的数据。
3. 工作负载完成后、将使用ONTAP 系统管理器和命令行界面收集存储效率统计信息。

观察结果

对于使用OMB工具生成的数据、我们发现空间节省~33%、存储效率比率为1.70：1。如下图所示、生成的数据所使用的逻辑空间为420.3 GB、用于存放数据的物理空间为281.7 GB。



```
shantanuCVOinstancenew::> df -h -S

Warning: The "-S" parameter is deprecated and may be removed in a future release. To show the efficiency ratio use "aggr show-efficiency"
         command.
Filesystem               used      total-saved    %total-saved    deduplicated    %deduplicated    compressed    %compressed  Vserver
/vol/vol0/             7319MB               0B              0%              0B               0%            0B             0%  shantanuCVOinstancenew-01
/vol/kafka_vol/         281GB            138GB             33%           138GB              33%            0B             0%  svm_shantanuCVOinstancenew
/vol/svm_shantanuCVOinstancenew_root/
                        660KB               0B              0%              0B               0%            0B             0%  svm_shantanuCVOinstancenew
3 entries were displayed.
```

```
                        Name of the Aggregate: aggr1
                Node where Aggregate Resides: shantanuCVOinstancenew-01
               Total Storage Efficiency Ratio: 1.70:1
Total Data Reduction Efficiency Ratio Without Snapshots: 1.70:1
Total Data Reduction Efficiency Ratio without snapshots and flexclones: 1.70:1
           Logical Space Used for All Volumes: 420.3GB
          Physical Space Used for All Volumes: 281.7GB
```

AWS中的性能概述和验证

基于AWS云中的性能对存储层挂载在NetApp NFS上的Kafka集群进行了基准测试。以下各节将介绍这些基准测试示例。

采用NetApp Cloud Volumes ONTAP 的AWS云中的Kafka (高可用性对和单节点)

采用NetApp Cloud Volumes ONTAP (HA对)的Kafka集群已通过AWS云性能基准测试。以下各节将介绍此基准测试。

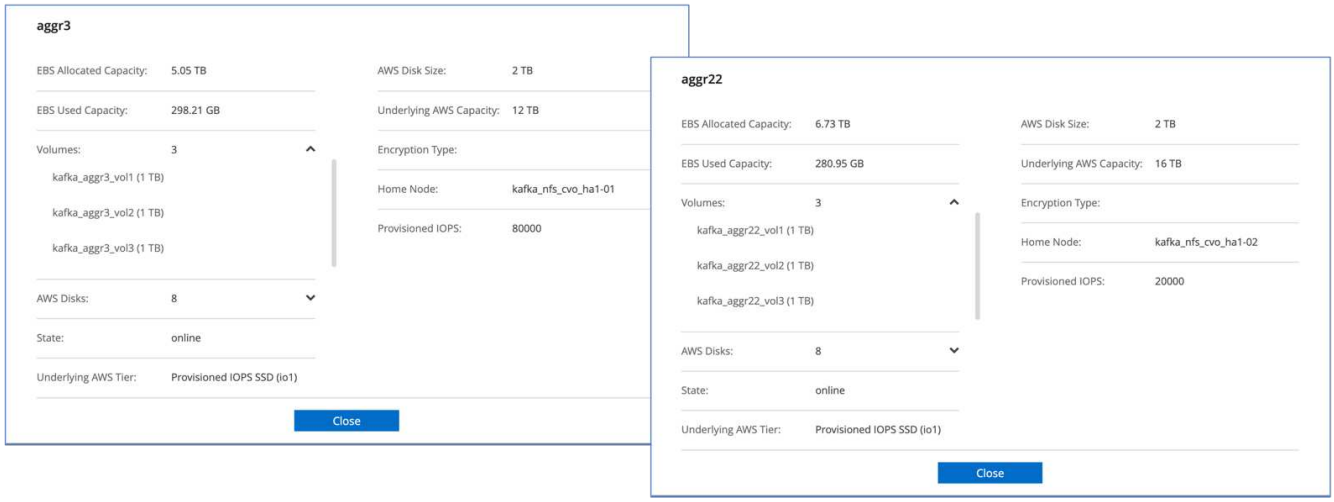
架构设置

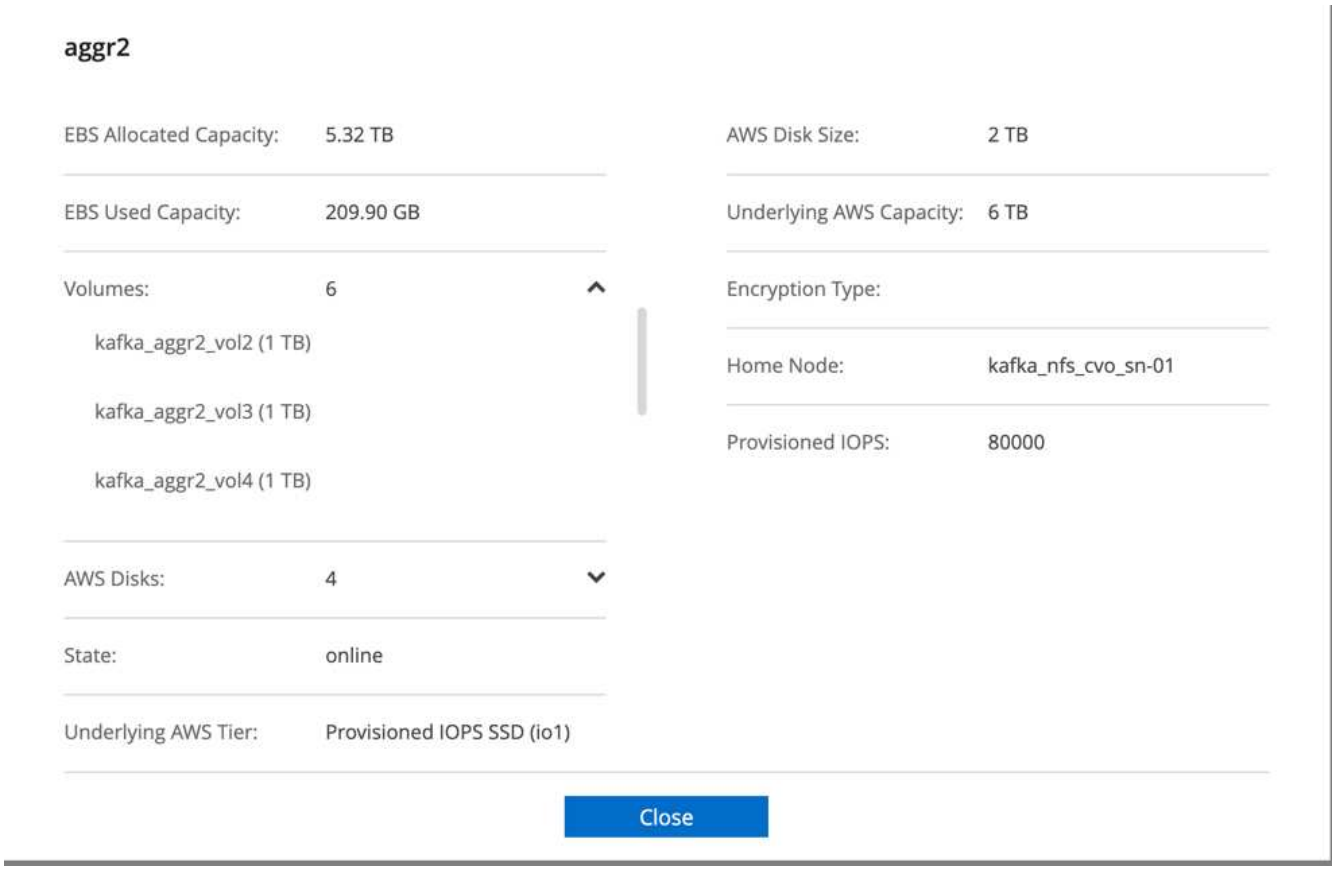
下表显示了使用NAS的Kafka集群的环境配置。

平台组件	环境配置
Kafka 3.2.3	• 3个Zookepers—T2.Small • 3个代理服务器—i3en.2xlarge • 1个Grafana—c5n.2xlarge • 4个生产者/使用者—c5n.2xlarge *
所有节点上的操作系统	RHEL8.6
NetApp Cloud Volumes ONTAP 实例	HA对实例—m5dn.12x插入x双节点单节点实例—m5dn.12x插入x 1个节点

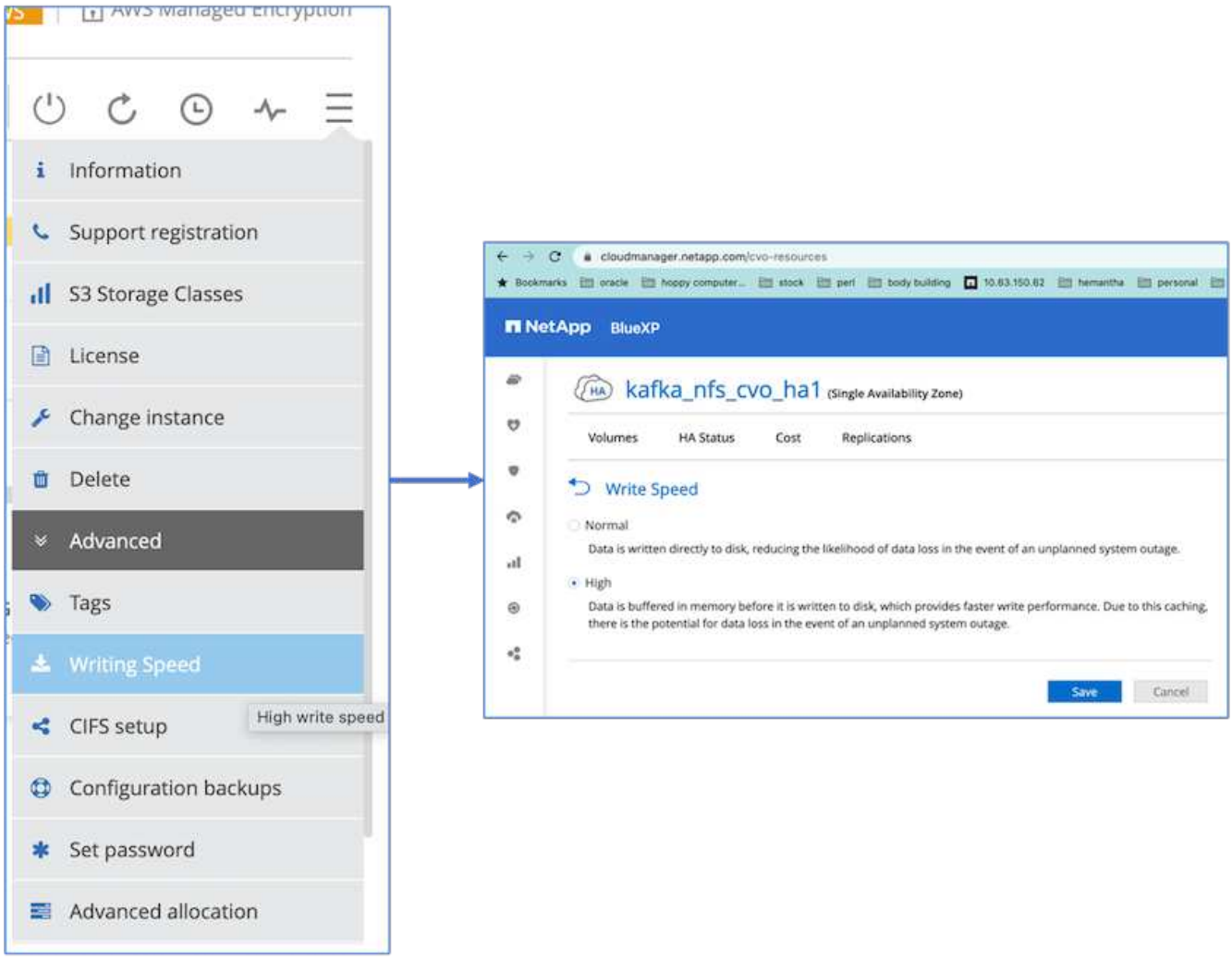
NetApp集群卷ONTAP 设置

1. 对于Cloud Volumes ONTAP HA对、我们在每个存储控制器的每个聚合上创建了两个聚合、其中包含三个卷。对于单个Cloud Volumes ONTAP 节点、我们会在一个聚合中创建六个卷。



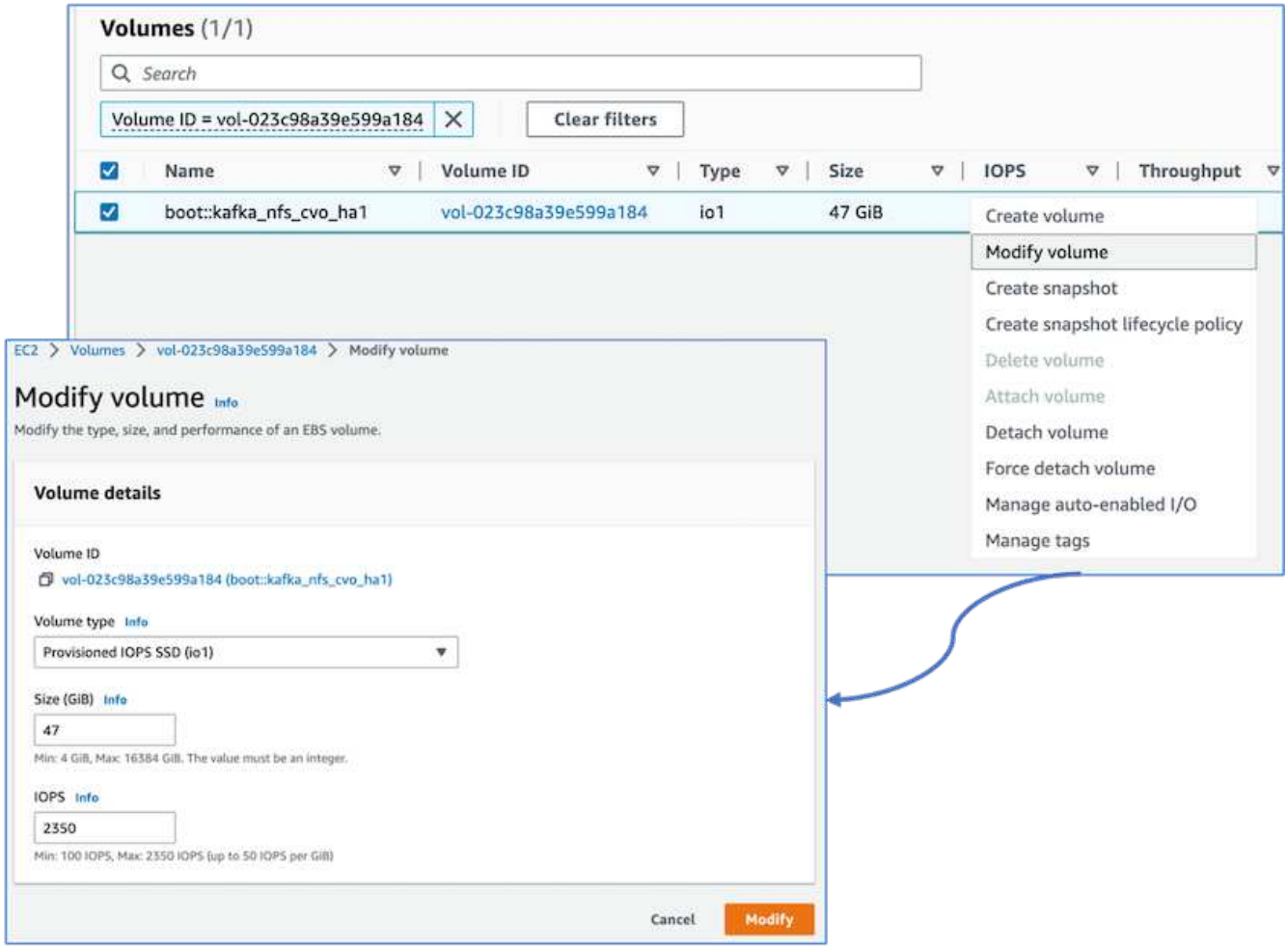


2. 为了提高网络性能、我们为HA对和单个节点启用了高速网络连接。



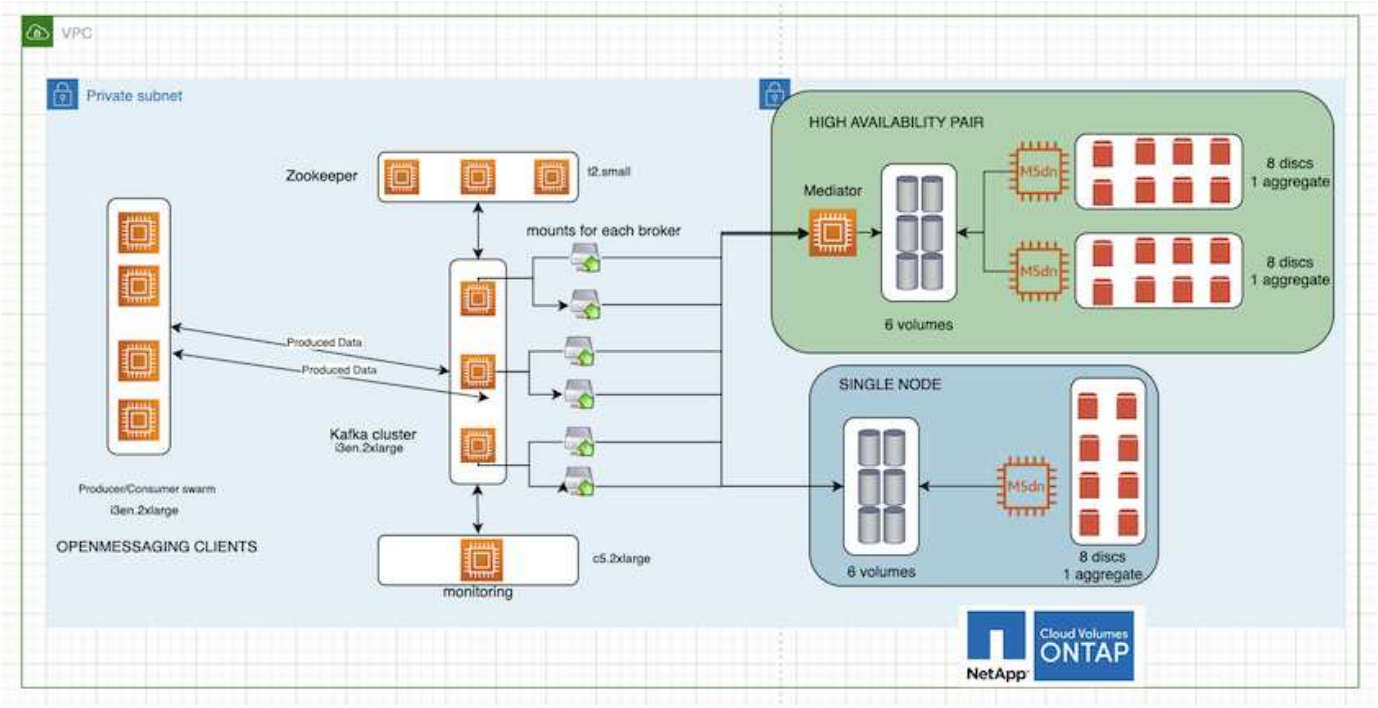
3. 我们注意到ONTAP NVRAM的IOPS较多、因此将Cloud Volumes ONTAP 根卷的IOPS更改为2350。Cloud Volumes ONTAP 中的根卷磁盘大小为47 GB。以下ONTAP 命令适用于HA对、同一步骤适用于单个节点。

```
statistics start -object vnvram -instance vnvram -counter
backing_store_iops -sample-id sample_555
kafka_nfs_cvo_ha1::*> statistics show -sample-id sample_555
Object: vnvram
Instance: vnvram
Start-time: 1/18/2023 18:03:11
End-time: 1/18/2023 18:03:13
Elapsed-time: 2s
Scope: kafka_nfs_cvo_ha1-01
    Counter                                                     Value
    -------------------------------- --------------------------------
    backing_store_iops                                           1479
Object: vnvram
Instance: vnvram
Start-time: 1/18/2023 18:03:11
End-time: 1/18/2023 18:03:13
Elapsed-time: 2s
Scope: kafka_nfs_cvo_ha1-02
    Counter                                                     Value
    -------------------------------- --------------------------------
    backing_store_iops                                           1210
2 entries were displayed.
kafka_nfs_cvo_ha1::*>
```



下图展示了基于NAS的Kafka集群的架构。

- *计算。*我们使用了一个三节点Kafka集群、其中三节点Zookeeper集合在专用服务器上运行。每个代理都通过一个专用LIF与Cloud Volumes ONTAP 实例上的一个卷具有两个NFS挂载点。
- *监控。*我们将两个节点用于Prometheus-Grafana组合。为了生成工作负载、我们使用了一个单独的三节点集群、该集群可能会生成此Kafka集群并将其占用。
- *存储。*我们使用了一个HA对Cloud Volumes ONTAP 实例、该实例上挂载了一个6 TB的GP3 AWS-EBS卷。然后、该卷会通过NFS挂载导出到Kafka代理。



OpenMessage基准配置

1. 为了提高NFS性能、我们需要在NFS服务器和NFS客户端之间建立更多的网络连接、这些连接可以使用nconnect来创建。运行以下命令、使用nconnect选项在代理节点上挂载NFS卷：

```
[root@ip-172-30-0-121 ~]# cat /etc/fstab
UUID=eaa1f38e-de0f-4ed5-a5b5-2fa9db43bb38/xfsdefaults00
/dev/nvme1n1 /mnt/data-1 xfs defaults,noatime,nodiscard 0 0
/dev/nvme2n1 /mnt/data-2 xfs defaults,noatime,nodiscard 0 0
172.30.0.233:/kafka_aggr3_vol1 /kafka_aggr3_vol1 nfs
defaults,nconnect=16 0 0
172.30.0.233:/kafka_aggr3_vol2 /kafka_aggr3_vol2 nfs
defaults,nconnect=16 0 0
172.30.0.233:/kafka_aggr3_vol3 /kafka_aggr3_vol3 nfs
defaults,nconnect=16 0 0
172.30.0.242:/kafka_aggr22_vol1 /kafka_aggr22_vol1 nfs
defaults,nconnect=16 0 0
172.30.0.242:/kafka_aggr22_vol2 /kafka_aggr22_vol2 nfs
defaults,nconnect=16 0 0
172.30.0.242:/kafka_aggr22_vol3 /kafka_aggr22_vol3 nfs
defaults,nconnect=16 0 0
[root@ip-172-30-0-121 ~]# mount -a
[root@ip-172-30-0-121 ~]# df -h
Filesystem                       Size  Used Avail Use% Mounted on
devtmpfs                          31G     0   31G   0% /dev
tmpfs                             31G  249M   31G   1% /run
tmpfs                             31G     0   31G   0% /sys/fs/cgroup
/dev/nvme0n1p2                    10G  2.8G  7.2G  28% /
/dev/nvme1n1                     2.3T  248G  2.1T  11% /mnt/data-1
/dev/nvme2n1                     2.3T  245G  2.1T  11% /mnt/data-2
172.30.0.233:/kafka_aggr3_vol1   1.0T   12G 1013G   2% /kafka_aggr3_vol1
172.30.0.233:/kafka_aggr3_vol2   1.0T  5.5G 1019G   1% /kafka_aggr3_vol2
172.30.0.233:/kafka_aggr3_vol3   1.0T  8.9G 1016G   1% /kafka_aggr3_vol3
172.30.0.242:/kafka_aggr22_vol1  1.0T  7.3G 1017G   1%
/kafka_aggr22_vol1
172.30.0.242:/kafka_aggr22_vol2  1.0T  6.9G 1018G   1%
/kafka_aggr22_vol2
172.30.0.242:/kafka_aggr22_vol3  1.0T  5.9G 1019G   1%
/kafka_aggr22_vol3
tmpfs                            6.2G     0  6.2G   0% /run/user/1000
[root@ip-172-30-0-121 ~]#
```

2. 在Cloud Volumes ONTAP 中检查网络连接。从单个Cloud Volumes ONTAP 节点使用以下ONTAP 命令。同一步骤也适用于Cloud Volumes ONTAP HA对。

```
Last login time: 1/20/2023 00:16:29
kafka_nfs_cvo_sn::> network connections active show -service nfs*
-fields remote-host
node                cid        vserver              remote-host
------------------- ---------- -------------------- ------------
```

```
kafka_nfs_cvo_sn-01 2315762628 svm_kafka_nfs_cvo_sn 172.30.0.121
kafka_nfs_cvo_sn-01 2315762629 svm_kafka_nfs_cvo_sn 172.30.0.121
kafka_nfs_cvo_sn-01 2315762630 svm_kafka_nfs_cvo_sn 172.30.0.121
kafka_nfs_cvo_sn-01 2315762631 svm_kafka_nfs_cvo_sn 172.30.0.121
kafka_nfs_cvo_sn-01 2315762632 svm_kafka_nfs_cvo_sn 172.30.0.121
kafka_nfs_cvo_sn-01 2315762633 svm_kafka_nfs_cvo_sn 172.30.0.121
kafka_nfs_cvo_sn-01 2315762634 svm_kafka_nfs_cvo_sn 172.30.0.121
kafka_nfs_cvo_sn-01 2315762635 svm_kafka_nfs_cvo_sn 172.30.0.121
kafka_nfs_cvo_sn-01 2315762636 svm_kafka_nfs_cvo_sn 172.30.0.121
kafka_nfs_cvo_sn-01 2315762637 svm_kafka_nfs_cvo_sn 172.30.0.121
kafka_nfs_cvo_sn-01 2315762639 svm_kafka_nfs_cvo_sn 172.30.0.72
kafka_nfs_cvo_sn-01 2315762640 svm_kafka_nfs_cvo_sn 172.30.0.72
kafka_nfs_cvo_sn-01 2315762641 svm_kafka_nfs_cvo_sn 172.30.0.72
kafka_nfs_cvo_sn-01 2315762642 svm_kafka_nfs_cvo_sn 172.30.0.72
kafka_nfs_cvo_sn-01 2315762643 svm_kafka_nfs_cvo_sn 172.30.0.72
kafka_nfs_cvo_sn-01 2315762644 svm_kafka_nfs_cvo_sn 172.30.0.72
kafka_nfs_cvo_sn-01 2315762645 svm_kafka_nfs_cvo_sn 172.30.0.72
kafka_nfs_cvo_sn-01 2315762646 svm_kafka_nfs_cvo_sn 172.30.0.72
kafka_nfs_cvo_sn-01 2315762647 svm_kafka_nfs_cvo_sn 172.30.0.72
kafka_nfs_cvo_sn-01 2315762648 svm_kafka_nfs_cvo_sn 172.30.0.72
kafka_nfs_cvo_sn-01 2315762649 svm_kafka_nfs_cvo_sn 172.30.0.121
kafka_nfs_cvo_sn-01 2315762650 svm_kafka_nfs_cvo_sn 172.30.0.121
kafka_nfs_cvo_sn-01 2315762651 svm_kafka_nfs_cvo_sn 172.30.0.121
kafka_nfs_cvo_sn-01 2315762652 svm_kafka_nfs_cvo_sn 172.30.0.121
kafka_nfs_cvo_sn-01 2315762653 svm_kafka_nfs_cvo_sn 172.30.0.121
kafka_nfs_cvo_sn-01 2315762656 svm_kafka_nfs_cvo_sn 172.30.0.223
kafka_nfs_cvo_sn-01 2315762657 svm_kafka_nfs_cvo_sn 172.30.0.223
kafka_nfs_cvo_sn-01 2315762658 svm_kafka_nfs_cvo_sn 172.30.0.223
kafka_nfs_cvo_sn-01 2315762659 svm_kafka_nfs_cvo_sn 172.30.0.223
kafka_nfs_cvo_sn-01 2315762660 svm_kafka_nfs_cvo_sn 172.30.0.223
kafka_nfs_cvo_sn-01 2315762661 svm_kafka_nfs_cvo_sn 172.30.0.223
kafka_nfs_cvo_sn-01 2315762662 svm_kafka_nfs_cvo_sn 172.30.0.223
kafka_nfs_cvo_sn-01 2315762663 svm_kafka_nfs_cvo_sn 172.30.0.223
kafka_nfs_cvo_sn-01 2315762664 svm_kafka_nfs_cvo_sn 172.30.0.223
kafka_nfs_cvo_sn-01 2315762665 svm_kafka_nfs_cvo_sn 172.30.0.223
kafka_nfs_cvo_sn-01 2315762666 svm_kafka_nfs_cvo_sn 172.30.0.223
kafka_nfs_cvo_sn-01 2315762667 svm_kafka_nfs_cvo_sn 172.30.0.72
kafka_nfs_cvo_sn-01 2315762668 svm_kafka_nfs_cvo_sn 172.30.0.72
kafka_nfs_cvo_sn-01 2315762669 svm_kafka_nfs_cvo_sn 172.30.0.72
kafka_nfs_cvo_sn-01 2315762670 svm_kafka_nfs_cvo_sn 172.30.0.72
kafka_nfs_cvo_sn-01 2315762671 svm_kafka_nfs_cvo_sn 172.30.0.72
kafka_nfs_cvo_sn-01 2315762672 svm_kafka_nfs_cvo_sn 172.30.0.72
kafka_nfs_cvo_sn-01 2315762673 svm_kafka_nfs_cvo_sn 172.30.0.223
kafka_nfs_cvo_sn-01 2315762674 svm_kafka_nfs_cvo_sn 172.30.0.223
kafka_nfs_cvo_sn-01 2315762676 svm_kafka_nfs_cvo_sn 172.30.0.121
kafka_nfs_cvo_sn-01 2315762677 svm_kafka_nfs_cvo_sn 172.30.0.223
```

```
kafka_nfs_cvo_sn-01 2315762678 svm_kafka_nfs_cvo_sn 172.30.0.223
kafka_nfs_cvo_sn-01 2315762679 svm_kafka_nfs_cvo_sn 172.30.0.223
48 entries were displayed.

kafka_nfs_cvo_sn::>
```

3. 我们使用以下Kafka `server.properties` 在Cloud Volumes ONTAP HA对的所有Kafka代理中。。`log.dirs` 每个代理的属性都不同、其余属性对于代理是通用的。对于Broker1、为 `log.dirs` 值如下：

```
[root@ip-172-30-0-121 ~]# cat /opt/kafka/config/server.properties
broker.id=0
advertised.listeners=PLAINTEXT://172.30.0.121:9092
#log.dirs=/mnt/data-1/d1,/mnt/data-1/d2,/mnt/data-1/d3,/mnt/data-
2/d1,/mnt/data-2/d2,/mnt/data-2/d3
log.dirs=/kafka_aggr3_vol1/broker1,/kafka_aggr3_vol2/broker1,/kafka_aggr
3_vol3/broker1,/kafka_aggr22_vol1/broker1,/kafka_aggr22_vol2/broker1,/ka
fka_aggr22_vol3/broker1
zookeeper.connect=172.30.0.12:2181,172.30.0.30:2181,172.30.0.178:2181
num.network.threads=64
num.io.threads=64
socket.send.buffer.bytes=102400
socket.receive.buffer.bytes=102400
socket.request.max.bytes=104857600
num.partitions=1
num.recovery.threads.per.data.dir=1
offsets.topic.replication.factor=1
transaction.state.log.replication.factor=1
transaction.state.log.min.isr=1
replica.fetch.max.bytes=524288000
background.threads=20
num.replica.alter.log.dirs.threads=40
num.replica.fetchers=20
[root@ip-172-30-0-121 ~]#
```

- 对于Broker2、为 `log.dirs` 属性值如下：

```
log.dirs=/kafka_aggr3_vol1/broker2,/kafka_aggr3_vol2/broker2,/kafka_a
ggr3_vol3/broker2,/kafka_aggr22_vol1/broker2,/kafka_aggr22_vol2/broke
r2,/kafka_aggr22_vol3/broker2
```

- 对于Broker3、为 `log.dirs` 属性值如下：

```
log.dirs=/kafka_aggr3_vol1/broker3,/kafka_aggr3_vol2/broker3,/kafka_a
ggr3_vol3/broker3,/kafka_aggr22_vol1/broker3,/kafka_aggr22_vol2/broke
r3,/kafka_aggr22_vol3/broker3
```

4. 对于单个Cloud Volumes ONTAP 节点、为Kafka `servers.properties` 与Cloud Volumes ONTAP HA对相同、但不包括 `log.dirs` 属性。
   - 对于Broker1、为 `log.dirs` 值如下：

```
log.dirs=/kafka_aggr2_vol1/broker1,/kafka_aggr2_vol2/broker1,/kafka_a
ggr2_vol3/broker1,/kafka_aggr2_vol4/broker1,/kafka_aggr2_vol5/broker1
,/kafka_aggr2_vol6/broker1
```

   - 对于Broker2、为 `log.dirs` 值如下：

```
log.dirs=/kafka_aggr2_vol1/broker2,/kafka_aggr2_vol2/broker2,/kafka_a
ggr2_vol3/broker2,/kafka_aggr2_vol4/broker2,/kafka_aggr2_vol5/broker2
,/kafka_aggr2_vol6/broker2
```

   - 对于Broker3、为 `log.dirs` 属性值如下：

```
log.dirs=/kafka_aggr2_vol1/broker3,/kafka_aggr2_vol2/broker3,/kafka_a
ggr2_vol3/broker3,/kafka_aggr2_vol4/broker3,/kafka_aggr2_vol5/broker3
,/kafka_aggr2_vol6/broker3
```

5. OMB中的工作负载配置了以下属性： `(/opt/benchmark/workloads/1-topic-100-partitions-1kb.yaml)`。

```
topics: 4
partitionsPerTopic: 100
messageSize: 32768
useRandomizedPayloads: true
randomBytesRatio: 0.5
randomizedPayloadPoolSize: 100
subscriptionsPerTopic: 1
consumerPerSubscription: 80
producersPerTopic: 40
producerRate: 1000000
consumerBacklogSizeGB: 0
testDurationMinutes: 5
```

。 `messageSize` 可能因使用情形而异。在性能测试中、我们使用了3 K。

我们使用OMB中的两个不同驱动程序Sync或Throughput在Kafka集群上生成工作负载。

- 用于Sync驱动程序属性的YAML文件如下所示 `(/opt/benchmark/driver- kafka/kafka-sync.yaml)`：

```
name: Kafka
driverClass:
io.openmessaging.benchmark.driver.kafka.KafkaBenchmarkDriver
# Kafka client-specific configuration
replicationFactor: 3
topicConfig: |
  min.insync.replicas=2
  flush.messages=1
  flush.ms=0
commonConfig: |

bootstrap.servers=172.30.0.121:9092,172.30.0.72:9092,172.30.0.223:909
2
producerConfig: |
  acks=all
  linger.ms=1
  batch.size=1048576
consumerConfig: |
  auto.offset.reset=earliest
  enable.auto.commit=false
  max.partition.fetch.bytes=10485760
```

- 用于吞吐量驱动程序属性的YAML文件如下所示 `(/opt/benchmark/driver- kafka/kafka-throughput.yaml)`：

```
name: Kafka
driverClass:
io.openmessaging.benchmark.driver.kafka.KafkaBenchmarkDriver
# Kafka client-specific configuration
replicationFactor: 3
topicConfig: |
  min.insync.replicas=2
commonConfig: |

bootstrap.servers=172.30.0.121:9092,172.30.0.72:9092,172.30.0.223:909
2
  default.api.timeout.ms=1200000
  request.timeout.ms=1200000
producerConfig: |
  acks=all
  linger.ms=1
  batch.size=1048576
consumerConfig: |
  auto.offset.reset=earliest
  enable.auto.commit=false
  max.partition.fetch.bytes=10485760
```

测试方法

1. Kafka集群是按照上述规范使用Terraform和Ansible配置的。Terraform用于使用适用于Kafka集群的AWS实例构建基础架构、Ansible在这些实例上构建Kafka集群。
2. 已使用上述工作负载配置和Sync驱动程序触发OMB工作负载。

```
Sudo bin/benchmark -drivers driver-kafka/kafka- sync.yaml workloads/1-
topic-100-partitions-1kb.yaml
```

3. 使用相同工作负载配置的吞吐量驱动程序触发了另一个工作负载。

```
sudo bin/benchmark -drivers driver-kafka/kafka-throughput.yaml
workloads/1-topic-100-partitions-1kb.yaml
```

观察结果

我们使用了两种不同类型的驱动程序来生成工作负载、以便对在NFS上运行的Kafka实例的性能进行基准测试。驱动程序之间的区别在于日志刷新属性。

对于Cloud Volumes ONTAP HA对：

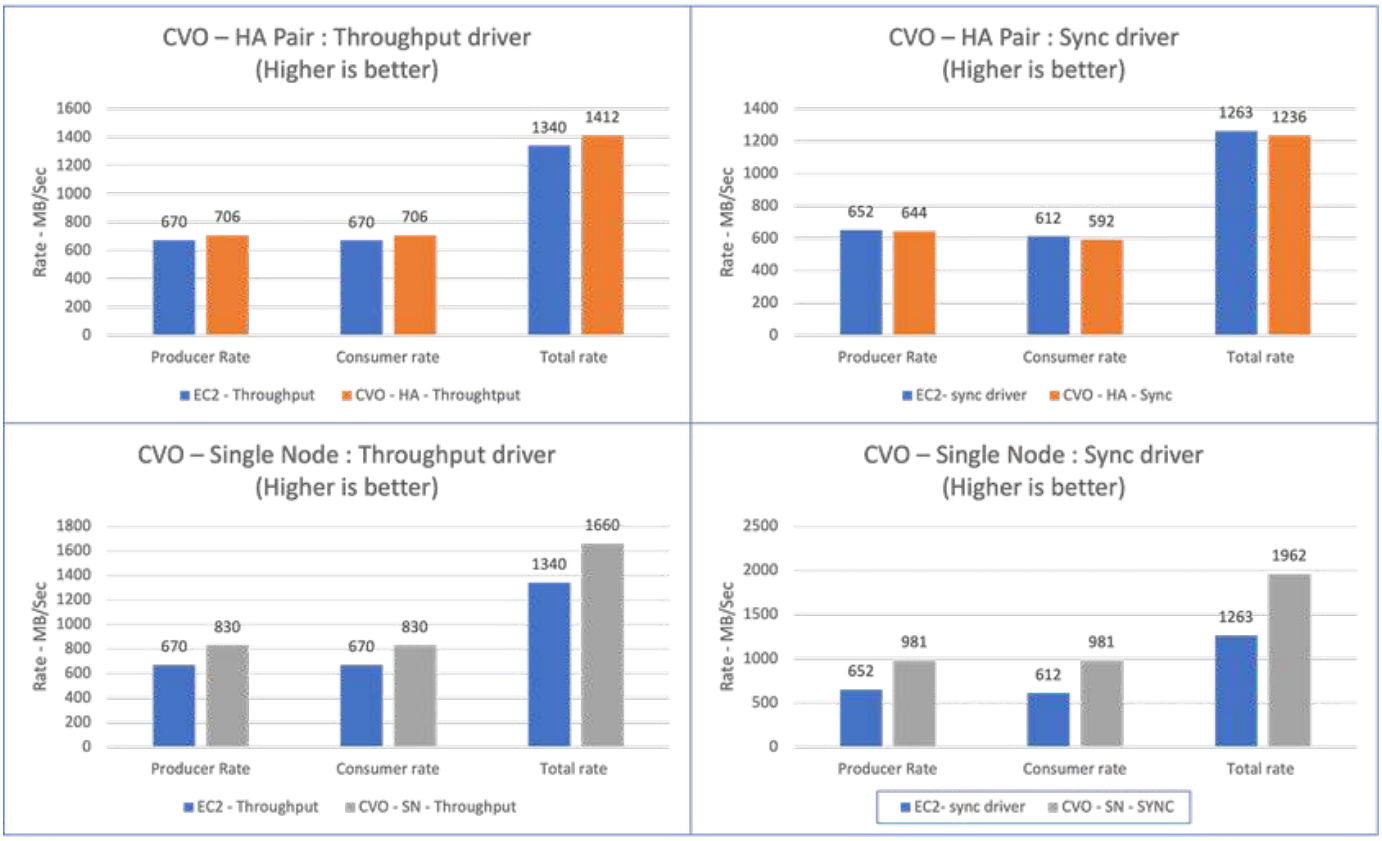
- Sync驱动程序一致生成的总吞吐量：~1236 MBps。
- 为吞吐量驱动程序生成的总吞吐量：峰值~1412 MBps。

对于单个Cloud Volumes ONTAP 节点：

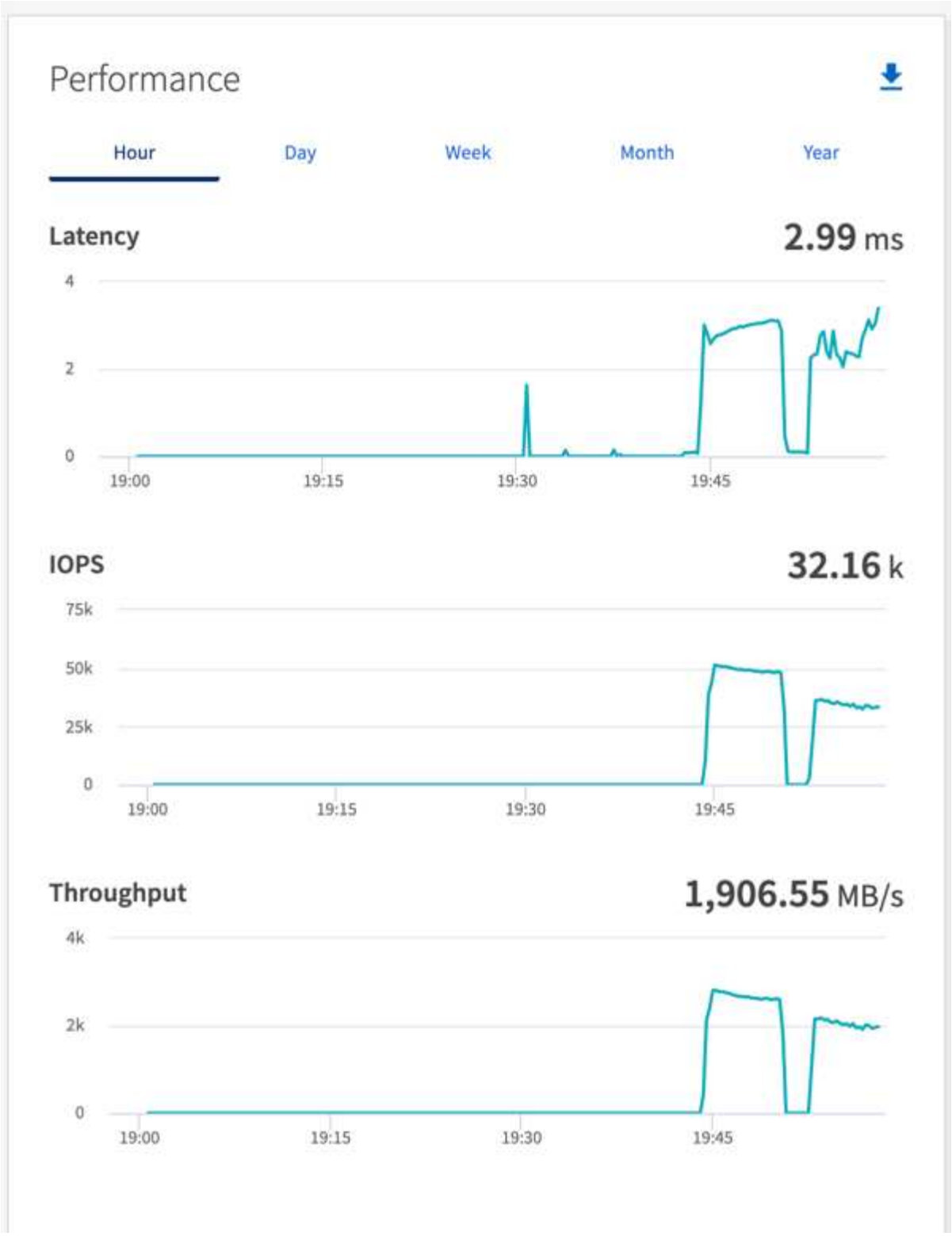
- Sync驱动程序一致生成的总吞吐量：~ 1962MBps。
- 吞吐量驱动程序生成的总吞吐量：峰值~1660MBps

同步驱动程序可以在日志即时转储到磁盘时生成一致的吞吐量、而吞吐量驱动程序则在将日志批量提交到磁盘时生成突发的吞吐量。

这些吞吐量数字是为给定的AWS配置生成的。为了满足更高的性能要求、可以进一步扩展和调整实例类型、以提高吞吐量。总吞吐量或总速率是生产者和使用者速率的组合。



在执行吞吐量或同步驱动程序基准测试时、请务必检查存储吞吐量。



AWS FSx for NetApp ONTAP中的性能概述和验证

在AWS FSx for NetApp ONTAP中、对NetApp NFS上挂载了存储层的Kafka集群进行了性能基准测试。以下各节将介绍这些基准测试示例。

AWS FSx for NetApp ONTAP中的Apache Kafka

网络文件系统(Network File System、NFS)是一种广泛用于存储大量数据的网络文件系统。在大多数企业中、数据越来越多地由Apache Kafka等流式应用程序生成。这些工作负载需要可扩展性、低延迟以及具有现代存储功能的强大数据采集架构。要实现实时分析并提供可指导行动的洞察力、需要一个设计完善且性能高的基础架构。

Kafka的设计支持POSIX兼容文件系统、并依靠文件系统来处理文件操作、但在NFS3文件系统上存储数据时、Kafka代理NFS客户端对文件操作的解释可能与XFS或ext4等本地文件系统不同。一个常见的示例是NFS愚蠢的重命名、该重命名导致Kafka代理在扩展集群和重新分配分区时失败。为了应对这一挑战、NetApp对开源Linux NFS客户端进行了更新、对RHEL8.7和RHEL9.1中的内容进行了一般更改、并从当前FSx for NetApp ONTAP版本ONTAP 9.12.1开始受支持。

Amazon FSx for NetApp ONTAP可在云中提供一个完全托管、可扩展且高性能的NFS文件系统。FSx for NetApp ONTAP上的Kafka数据可以进行扩展、以处理大量数据并确保容错。NFS可为关键和敏感数据集提供集中式存储管理和数据保护。

通过这些增强功能、AWS客户可以在AWS计算服务上运行Kafka工作负载时利用FSx for NetApp ONTAP。这些优势包括：
*降低CPU利用率以缩短I/O等待时间
* Kafka代理恢复时间更快。
*可靠性和效率。
*可扩展性和性能。
*多可用性区域可用性。
*数据保护。

AWS FSx for NetApp ONTAP中的性能概述和验证

基于AWS云中的性能对存储层挂载在NetApp NFS上的Kafka集群进行了基准测试。以下各节将介绍这些基准测试示例。

Kafka位于AWS FSx for NetApp ONTAP中

采用AWS FSx for NetApp ONTAP的Kafka集群已通过AWS云中的性能基准测试。以下各节将介绍此基准测试。

架构设置

下表显示了使用AWS FSx for NetApp ONTAP的Kafka集群的环境配置。

平台组件	环境配置
Kafka 3.2.3	• 3个Zookepers—T2.Small • 3个代理服务器—i3en.2xlarge • 1个Grafana—c5n.2xlarge • 4个生产者/使用者—c5n.2xlarge *
所有节点上的操作系统	RHEL8.6
AWS FSx for NetApp ONTAP	多可用性(AZ)、吞吐量为4 GB/秒、IOPS为160000次

NetApp FSx for NetApp ONTAP设置

1. 在初始测试中、我们为NetApp ONTAP文件系统创建了一个FSx、容量为2 TB、吞吐量为400、000 IOPS、每秒2 GB。

```
[root@ip-172-31-33-69 ~]# aws fsx create-file-system --region us-east-2
--storage-capacity 2048 --subnet-ids <desired subnet 1> subnet-<desired
subnet 2> --file-system-type ONTAP --ontap-configuration
DeploymentType=MULTI_AZ_HA_1,ThroughputCapacity=2048,PreferredSubnetId=<
desired primary subnet>,FsxAdminPassword=<new
password>,DiskIopsConfiguration="{Mode=USER_PROVISIONED,Iops=40000"}
```

   在本示例中、我们将通过AWS命令行界面部署FSx for NetApp ONTAP。您需要根据需要在环境中进一步自定义此命令。此外、FSx for NetApp ONTAP还可以通过AWS控制台进行部署和管理、以减少命令行输入、获得更轻松、更简化的部署体验。

   文档在FSx for NetApp ONTAP中、测试区域(US-East-1)中2 GB/秒吞吐量文件系统可实现的最大IOPS为80、000次IOPS。FSx for NetApp ONTAP文件系统的总最大IOPS为160、000次IOPS、需要部署4 GB/秒吞吐量才能达到此目的、我们将在本文档后面进行演示。

   有关FSx for NetApp ONTAP性能规格的详细信息、请随时访问AWS FSx for NetApp ONTAP文档、网址为： https://docs.aws.amazon.com/fsx/latest/ONTAPGuide/performance.html 。

   有关FSx "crea-File-system"的详细命令行语法、请参见： https://docs.aws.amazon.com/cli/latest/reference/fsx/create-file-system.html

   例如、您可以指定特定的KMS密钥、而不是在未指定KMS密钥时使用的默认AWS FSx主密钥。

2. 创建FSx for NetApp ONTAP文件系统时、请等到JSON返回中的"LifeCycle (生命周期)"状态更改为"Available (可用)"、然后按如下所示描述文件系统：

```
[root@ip-172-31-33-69 ~]# aws fsx describe-file-systems  --region us-
east-1 --file-system-ids fs-02ff04bab5ce01c7c
```

3. 使用fsxadmin用户登录到FSx for NetApp ONTAP SSH以验证凭据：
   FSxadmin是创建时FSx for NetApp ONTAP文件系统的默认管理员帐户。fsxadmin的密码是我们在步骤1中完成的首次在AWS控制台中或使用AWS命令行界面创建文件系统时配置的密码。

```
[root@ip-172-31-33-69 ~]# ssh fsxadmin@198.19.250.244
The authenticity of host '198.19.250.244 (198.19.250.244)' can't be
established.
ED25519 key fingerprint is
SHA256:mgCyRXJfWRc2d/jOjFbMBsUcYOWjxoIky0ltHvVDL/Y.
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '198.19.250.244' (ED25519) to the list of
known hosts.
(fsxadmin@198.19.250.244) Password:

This is your first recorded login.
```

4. 验证凭据后、在FSx for NetApp ONTAP文件系统上创建Storage Virtual Machine

```
[root@ip-172-31-33-69 ~]# aws fsx --region us-east-1 create-storage-
virtual-machine --name svmkafkatest --file-system-id fs-
02ff04bab5ce01c7c
```

Storage Virtual Machine (SVM)是一种孤立的文件服务器、具有自己的管理凭据和端点、用于管理和访问FSx for NetApp ONTAP卷中的数据、并提供FSx for NetApp ONTAP多租户功能。

5. 配置主Storage Virtual Machine后、通过SSH连接到新创建的FSx for NetApp ONTAP文件系统、然后使用以下示例命令在Storage Virtual Machine中创建卷、同样、我们会为此验证创建6个卷。根据我们的验证、保留默认成分卷(8)或更少的成分卷、这样可以提高Kafka的性能。

```
FsxId02ff04bab5ce01c7c::*> volume create -volume kafkafsxN1 -state
online -policy default -unix-permissions ---rwxr-xr-x -junction-active
true -type RW -snapshot-policy none  -junction-path /kafkafsxN1 -aggr
-list aggr1
```

6. 我们需要在卷中增加容量以进行测试。将卷的大小扩展到2 TB、然后挂载到接合路径上。

```
FsxId02ff04bab5ce01c7c::*> volume size -volume kafkafsxN1 -new-size +2TB
vol size: Volume "svmkafkatest:kafkafsxN1" size set to 2.10t.

FsxId02ff04bab5ce01c7c::*> volume size -volume kafkafsxN2 -new-size +2TB
vol size: Volume "svmkafkatest:kafkafsxN2" size set to 2.10t.

FsxId02ff04bab5ce01c7c::*> volume size -volume kafkafsxN3 -new-size +2TB
vol size: Volume "svmkafkatest:kafkafsxN3" size set to 2.10t.

FsxId02ff04bab5ce01c7c::*> volume size -volume kafkafsxN4 -new-size +2TB
```

```
vol size: Volume "svmkafkatest:kafkafsxN4" size set to 2.10t.

FsxId02ff04bab5ce01c7c::*> volume size -volume kafkafsxN5 -new-size +2TB
vol size: Volume "svmkafkatest:kafkafsxN5" size set to 2.10t.

FsxId02ff04bab5ce01c7c::*> volume size -volume kafkafsxN6 -new-size +2TB
vol size: Volume "svmkafkatest:kafkafsxN6" size set to 2.10t.

FsxId02ff04bab5ce01c7c::*> volume show -vserver svmkafkatest -volume *
Vserver   Volume       Aggregate    State      Type       Size
Available Used%
--------- ------------ ------------ ---------- ---- ----------
---------- -----
svmkafkatest
          kafkafsxN1   -            online     RW       2.10TB
1.99TB    0%
svmkafkatest
          kafkafsxN2   -            online     RW       2.10TB
1.99TB    0%
svmkafkatest
          kafkafsxN3   -            online     RW       2.10TB
1.99TB    0%
svmkafkatest
          kafkafsxN4   -            online     RW       2.10TB
1.99TB    0%
svmkafkatest
          kafkafsxN5   -            online     RW       2.10TB
1.99TB    0%
svmkafkatest
          kafkafsxN6   -            online     RW       2.10TB
1.99TB    0%
svmkafkatest
          svmkafkatest_root
                       aggr1        online     RW          1GB
968.1MB    0%
7 entries were displayed.

FsxId02ff04bab5ce01c7c::*> volume mount -volume kafkafsxN1 -junction
-path /kafkafsxN1

FsxId02ff04bab5ce01c7c::*> volume mount -volume kafkafsxN2 -junction
-path /kafkafsxN2

FsxId02ff04bab5ce01c7c::*> volume mount -volume kafkafsxN3 -junction
-path /kafkafsxN3
```

```
FsxId02ff04bab5ce01c7c::*> volume mount -volume kafkafsxN4 -junction
-path /kafkafsxN4

FsxId02ff04bab5ce01c7c::*> volume mount -volume kafkafsxN5 -junction
-path /kafkafsxN5

FsxId02ff04bab5ce01c7c::*> volume mount -volume kafkafsxN6 -junction
-path /kafkafsxN6
```

在FSx for NetApp ONTAP中、可以对卷进行精简配置。在我们的示例中、扩展卷总容量超过文件系统总容量、因此我们需要扩展文件系统总容量、以便解锁额外配置的卷容量、我们将在下一步演示这一点。

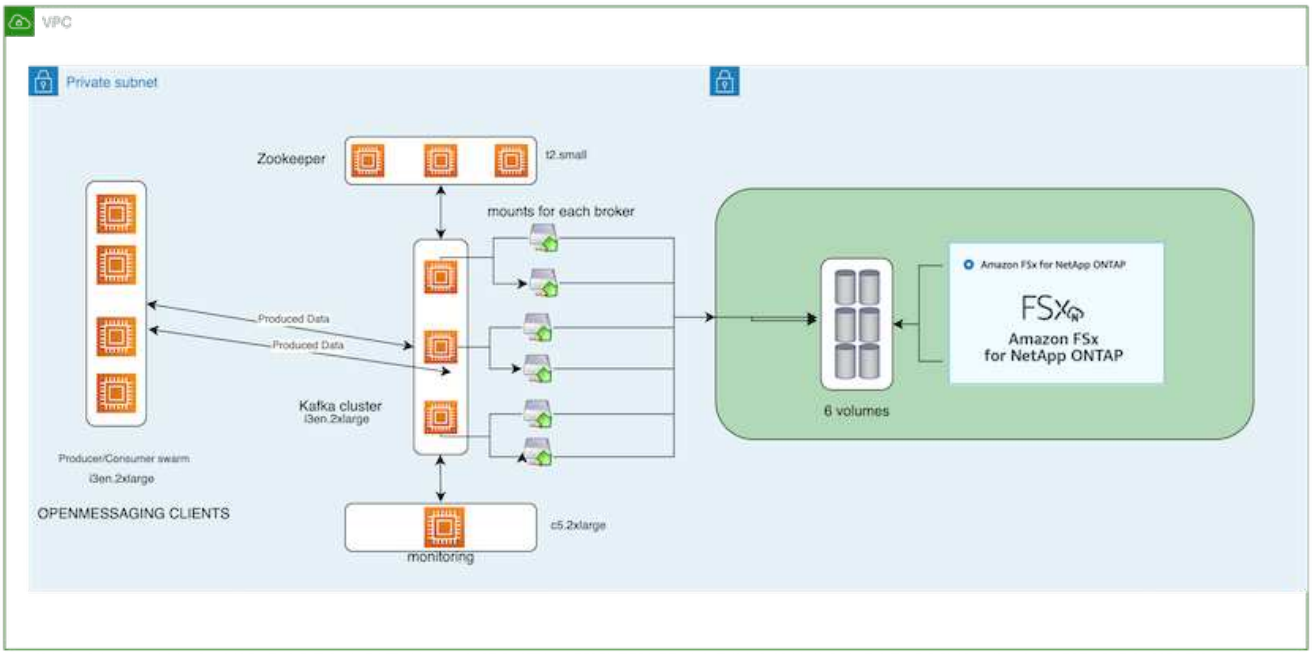
7. 接下来、为了提高性能和容量、我们将FSx的NetApp ONTAP吞吐量容量从2 GB/秒扩展到4 GB/秒、将IOPS扩展到160000、并将容量扩展到5 TB

```
[root@ip-172-31-33-69 ~]# aws fsx update-file-system --region us-east-1
--storage-capacity 5120 --ontap-configuration
'ThroughputCapacity=4096,DiskIopsConfiguration={Mode=USER_PROVISIONED,Io
ps=160000}' --file-system-id fs-02ff04bab5ce01c7c
```

有关FSx "update-file-system"的详细命令行语法、请参见：
https://docs.aws.amazon.com/cli/latest/reference/fsx/update-file-system.html

8. FSx for NetApp ONTAP卷使用nconnect和默认选项挂载在Kafka代理中

下图显示了基于Kafka集群的FSx for NetApp ONTAP的最终架构：



- 计算。我们使用了一个三节点Kafka集群、其中一个三节点Zookeer集合运行在专用服务器上。每个代理都有六个NFS挂载点、指向FSx for NetApp ONTAP实例上的六个卷。

- 监控。我们将两个节点用于Prometheus-Grafana组合。为了生成工作负载、我们使用了一个单独的三节点集群、该集群可能会生成此Kafka集群并将其占用。
- 存储。我们使用FSx for NetApp ONTAP、其中已挂载六个2 TB卷。然后、使用NFS挂载将卷导出到Kafka代理。FSx for NetApp ONTAP卷使用Kafka代理中的16个nconnect会话和默认选项进行挂载。

OpenMessage基准测试配置。

我们使用了与NetApp Cloud Volumes ONTAP相同的配置、其详细信息如下所示- https://docs.netapp.com/us-en/netapp-solutions/data-analytics/kafka-nfs-performance-overview-and-validation-in-aws.html#architectural-setup

测试方法

1. 按照上述规范、我们使用terraform和Ans得 来配置Kafka集群。Terraform用于使用适用于Kafka集群的AWS实例构建基础架构、而Ans可 在这些实例上构建Kafka集群。
2. 已使用上述工作负载配置和Sync驱动程序触发OMB工作负载。

```
sudo bin/benchmark -drivers driver-kafka/kafka-sync.yaml workloads/1-topic-100-partitions-1kb.yaml
```

3. 使用相同工作负载配置的吞吐量驱动程序触发了另一个工作负载。

```
sudo bin/benchmark -drivers driver-kafka/kafka-throughput.yaml workloads/1-topic-100-partitions-1kb.yaml
```

观察结果

我们使用了两种不同类型的驱动程序来生成工作负载、以便对在NFS上运行的Kafka实例的性能进行基准测试。驱动程序之间的区别在于日志刷新属性。

对于Kafka复制因子1和FSx for NetApp ONTAP：

- Sync驱动程序一致生成的总吞吐量：~ 3218 Mbps、峰值性能(~ 3352 Mbps)。
- 吞吐量驱动程序一致生成的总吞吐量：~ 3639 Mbps、峰值性能(~ 3908 Mbps)。

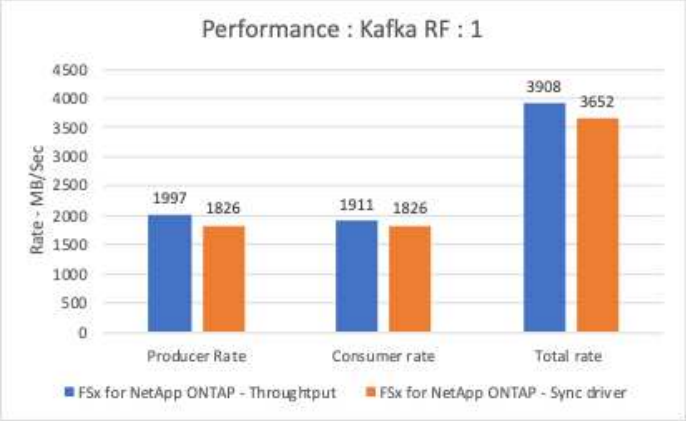
对于复制因子为3的Kafka和FSx for NetApp ONTAP：

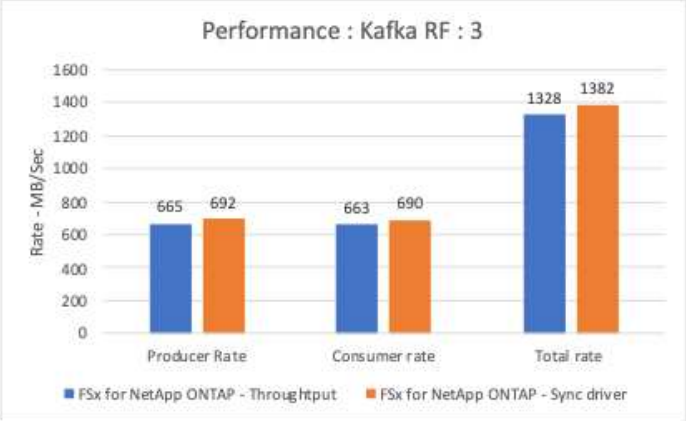
- Sync驱动程序一致生成的总吞吐量：~ 1252 Mbps、峰值性能(~ 1382 Mbps)。
- 吞吐量驱动程序一致生成的总吞吐量：~ 1218 MBps、峰值性能(以~ 1328 MBps为单位)。

在Kafka复制因子3中、读取和写入操作在FSx for NetApp ONTAP上发生了三次；在Kafka复制因子1中、读取和写入操作在FSx for NetApp ONTAP上发生了一次、因此、在这两种验证中、 我们可以达到4 GB/秒的最大吞吐量。

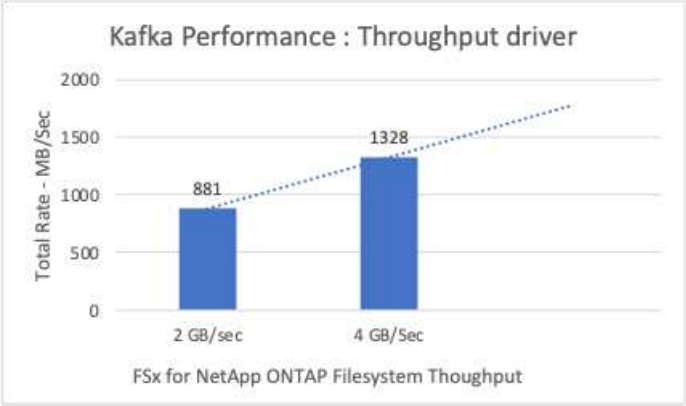
同步驱动程序可以在日志即时转储到磁盘时生成一致的吞吐量、而吞吐量驱动程序则在将日志批量提交到磁盘时生成突发的吞吐量。

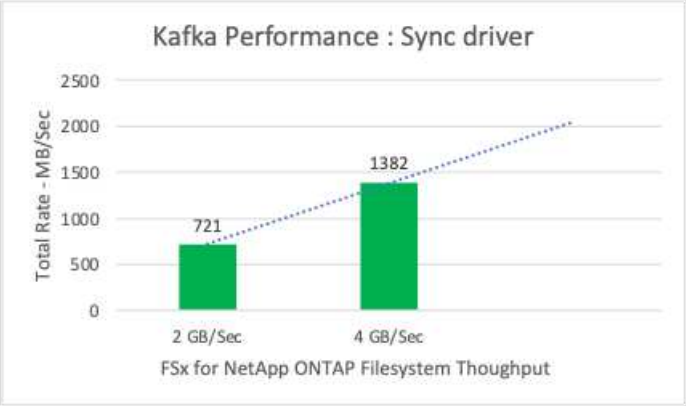
这些吞吐量数字是为给定的AWS配置生成的。为了满足更高的性能要求、可以进一步扩展和调整实例类型、以提高吞吐量。总吞吐量或总速率是生产者和使用者速率的组合。



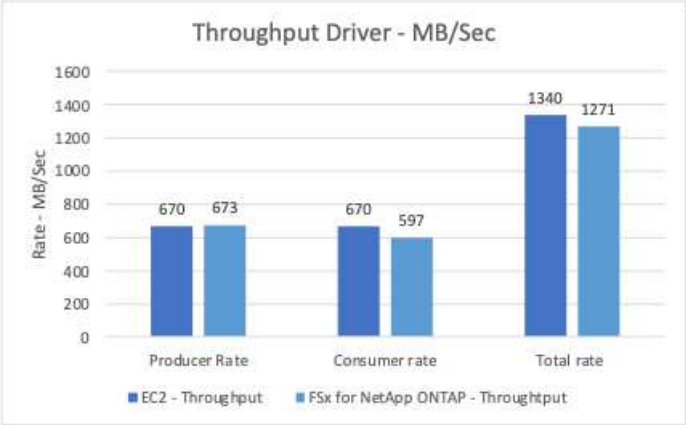


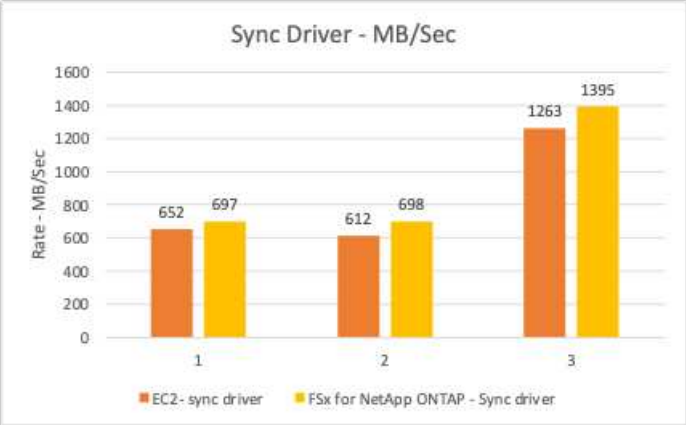
下图显示了NetApp ONTAP的2 GB/秒FSx和Kafka复制因子3的4 GB/秒性能。复制因子3在FSx for NetApp ONTAP存储上执行三次读取和写入操作。吞吐量驱动程序的总速率为881 MB/秒、在NetApp ONTAP文件系统的2 GB/秒FSx上执行读取和写入Kafka操作的速率约为2.64 GB/秒、吞吐量驱动程序的总速率为1328 MB/秒、执行读取和写入Kafka操作的速率约为3.98 GB/秒。Kafka的性能是线性的、可根据FSx for NetApp ONTAP吞吐量进行扩展。





下图显示了EC2实例与FSx for NetApp ONTAP之间的性能(Kafka复制因子：3)

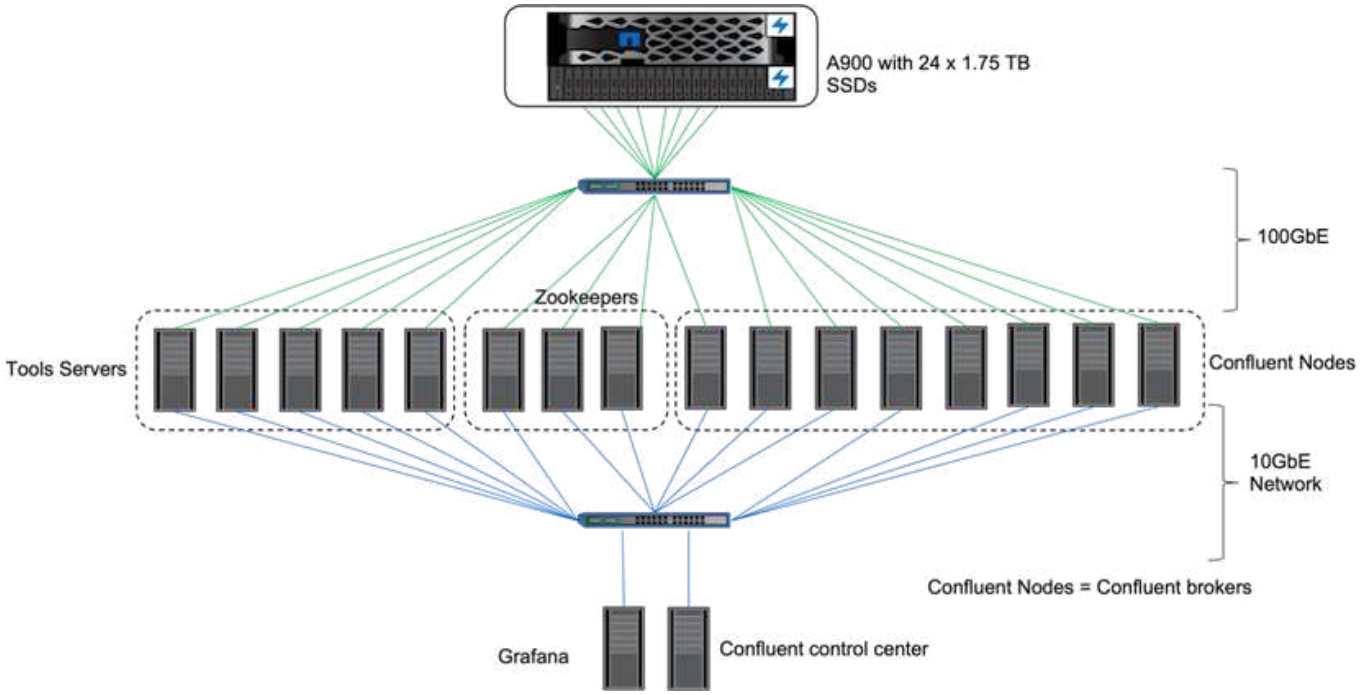




使用AFF A900内部部署进行性能概述和验证

在内部部署中、我们使用NetApp AFF A900存储控制器和ONTAP 9.12.1RC1来验证Kafka集群的性能和扩展能力。我们使用的测试平台与先前使用ONTAP 和AFF 的分层存储最佳实践中的测试平台相同。

我们使用Confluent Kafka 6.2.0对AFF A900进行了评估。集群具有八个代理节点和三个Zookeeper节点。在性能测试中、我们使用了五个OMB辅助节点。



存储配置

我们使用NetApp FlexGroup实例为日志目录提供了一个命名空间、从而简化了恢复和配置。我们使用NFSv4.1和pNFS为日志段数据提供直接路径访问。

客户端调整

每个客户端都使用以下命令挂载FlexGroup 实例。

```
mount -t nfs -o vers=4.1,nconnect=16 172.30.0.121:/kafka_vol01
/data/kafka_vol01
```

此外、我们还增加了 `max_session_slots`` 默认值 `64` to `180`。这与ONTAP 中的默认会话插槽限制匹配。

Kafka代理调整

为了最大程度地提高测试系统中的吞吐量、我们显著增加了某些关键线程池的默认参数。对于大多数配置、我们建议遵循Confluent Kafka最佳实践。此调整用于最大程度地提高存储未处理I/O的并发性。可以根据代理的计算资源和存储属性调整这些参数。

```
num.io.threads=96
num.network.threads=96
background.threads=20
num.replica.alter.log.dirs.threads=40
num.replica.fetchers=20
queued.max.requests=2000
```

工作负载生成器测试方法

我们使用的OMB配置与对吞吐量驱动程序和主题配置进行云测试相同。

1. 已在AFF 集群上使用Ansible配置FlexGroup 实例。

```
---
- name: Set up kafka broker processes
  hosts: localhost
  vars:
    ntap_hostname: ‘hostname’
    ntap_username: 'user'
    ntap_password: 'password'
    size: 10
    size_unit: tb
    vserver: vs1
    state: present
    https: true
    export_policy: default
    volumes:
      - name: kafka_fg_vol01
        aggr: ["aggr1_a", "aggr2_a", “aggr1_b”, “aggr2_b”]
        path: /kafka_fg_vol01
  tasks:
    - name: Edit volumes
      netapp.ontap.na_ontap_volume:
        state: "{{ state }}"
        name: "{{ item.name }}"
        aggr_list: "{{ item.aggr }}"
        aggr_list_multiplier: 8
        size: "{{ size }}"
        size_unit: "{{ size_unit }}"
        vserver: "{{ vserver }}"
        snapshot_policy: none
        export_policy: default
        junction_path: "{{ item.path }}"
        qos_policy_group: none
        wait_for_completion: True
        hostname: "{{ ntap_hostname }}"
        username: "{{ ntap_username }}"
        password: "{{ ntap_password }}"
        https: "{{ https }}"
        validate_certs: false
      connection: local
      with_items: "{{ volumes }}"
```

2. 已在ONTAP SVM上启用pNFS。

```
vserver modify -vserver vs1 -v4.1-pnfs enabled -tcp-max-xfer-size 262144
```

3. 此工作负载是使用与Cloud Volumes ONTAP 相同的工作负载配置通过吞吐量驱动程序触发的。请参见第节"[稳定状态性能]"。工作负载使用的复制系数为3、这意味着在NFS中维护了三个日志段副本。

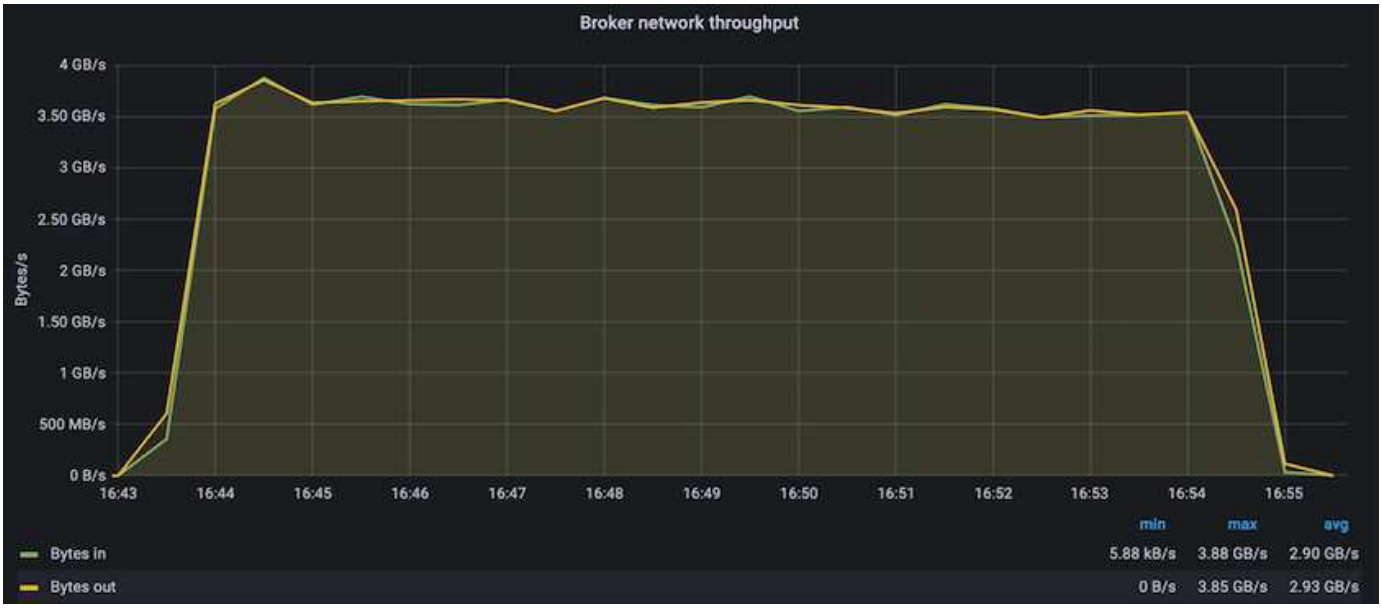
```
sudo bin/benchmark --drivers driver-kafka/kafka-throughput.yaml
workloads/1-topic-100-partitions-1kb.yaml
```

4. 最后、我们使用积压数据完成了测量、以衡量消费者是否能够跟上最新消息的步伐。OMB通过在测量开始期间暂停使用者来构建积压。这会产生三个不同的阶段：创建积压(仅限生产商的流量)、积压耗尽(消费者在一个主题中遇到未完成的事件时会遇到大量耗时的阶段)和稳定状态。请参见第节"探索存储限制"了解更多信息。

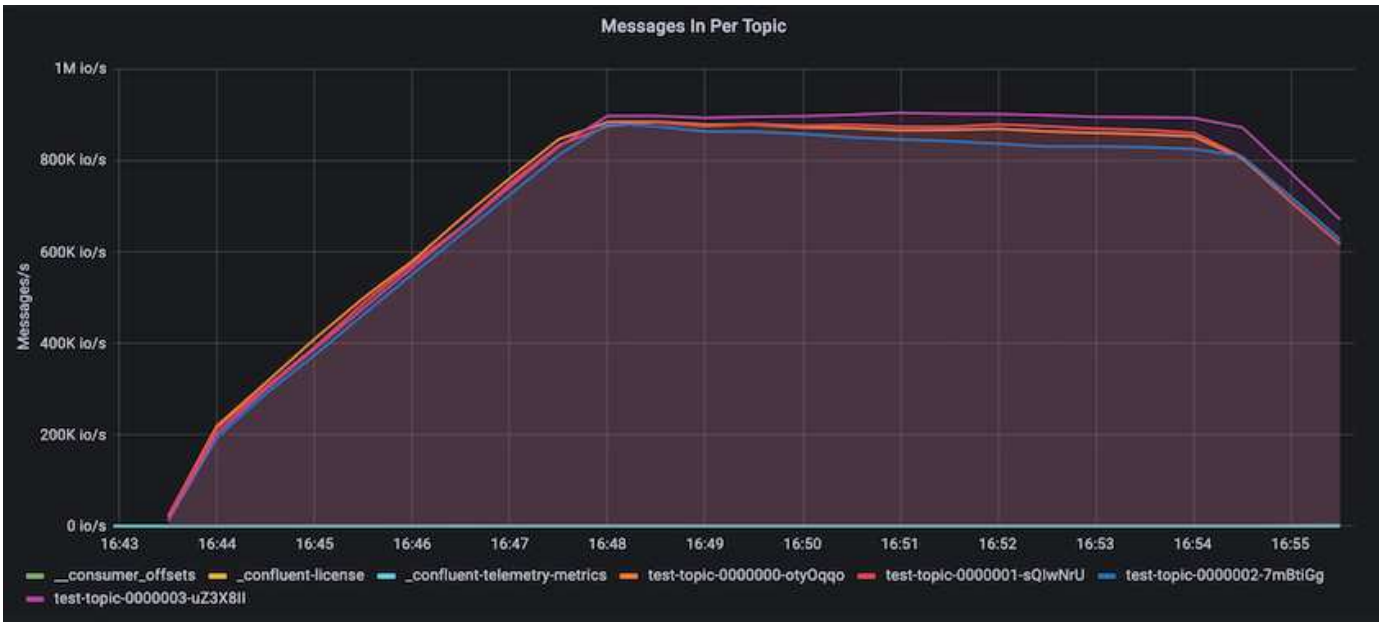
稳定状态性能

我们使用Open消息 基准测试对AFF A900进行了评估、以提供与AWS中的Cloud Volumes ONTAP 和AWS中的DAS类似的比较结果。所有性能值均表示生产商和消费者级别的Kafka-cluster吞吐量。

借助Confluent Kafka和AFF A900实现稳定的状态性能、生产者和使用者的平均吞吐量均超过3.4 GBps。在整个Kafka集群中、此消息超过340万条。通过直观地显示BrokerTopicMetrics的持续吞吐量(以字节/秒为单位)、我们可以看到AFF A900所支持的出色稳定状态性能和流量。



这与每个主题所传送消息的视图非常一致。下图按主题显示了细分情况。在测试的配置中、我们在四个主题中看到了每个主题近900、000条消息。

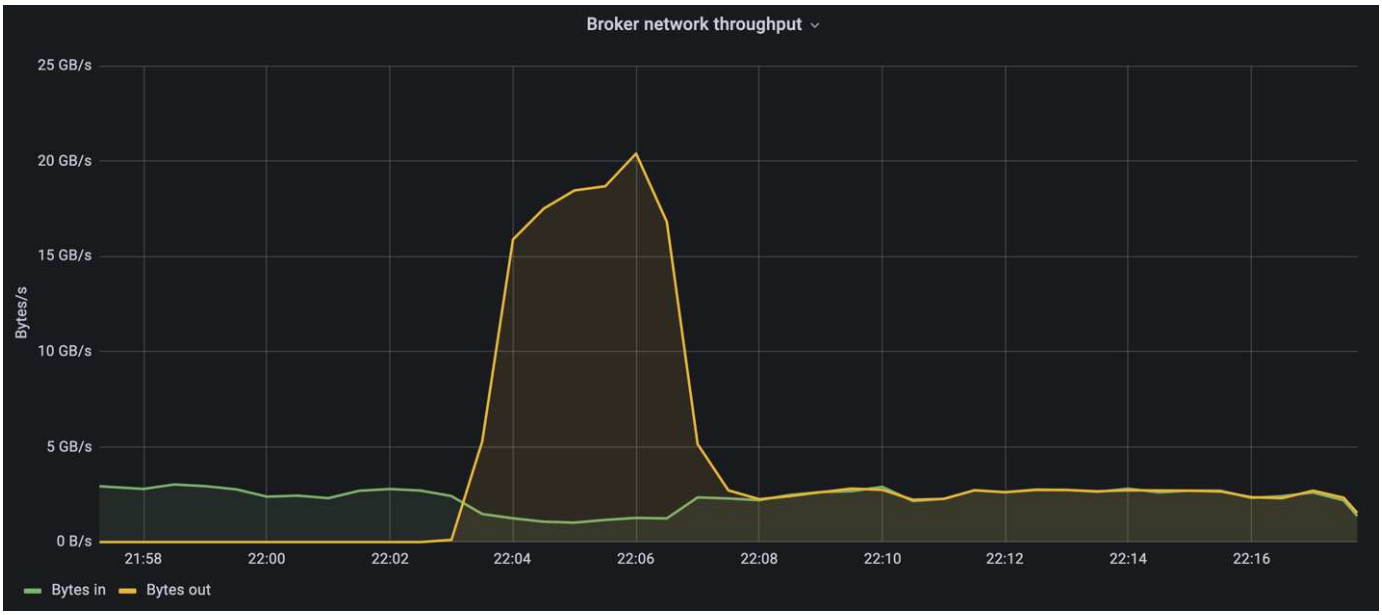


极致性能并探索存储限制

对于AFF 、我们还使用积压功能对OMB进行了测试。在Kafka集群中创建积压事件时、积压功能会暂停使用者订阅。在此阶段、仅会发生生成方流量、此流量会生成提交到日志的事件。这最能模拟批处理或脱机分析工作流；在这些工作流中、用户订阅会启动、并且必须读取已从代理缓存中逐出的历史数据。

为了了解此配置中对使用者吞吐量的存储限制、我们测量了纯生产者阶段、以了解A900可以吸收多少写入流量。请参见下一节"[规模估算指南]了解如何利用这些数据。

在本次测量中、我们发现、在仅用于生产商的部分、峰值吞吐量会突破A900性能的限制(此时、其他代理资源不会饱和地为生产商和消费者提供服务)。



我们将此度量值的消息大小增加到16k、以限制每条消息的开销、并最大程度地提高NFS挂载点的存储吞吐量。

```
messageSize: 16384
consumerBacklogSizeGB: 4096
```

Confluent Kafka集群的生产商吞吐量峰值为4.03 GBps。

```
18:12:23.833 [main] INFO WorkloadGenerator - Pub rate 257759.2 msg/s /
4027.5 MB/s | Pub err     0.0 err/s …
```

在OMB完成事件积压填充后、使用者流量将重新启动。在对积压量进行测量期间、我们在所有主题中观察到消费者峰值吞吐量超过20 Gbps。存储OMB日志数据的NFS卷的总吞吐量接近~30Gbps。

规模估算指南

Amazon Web Services提供了 "规模估算指南" 用于Kafka集群规模估算和扩展。

此规模估算为确定Kafka集群的存储吞吐量要求提供了一个有用的公式：

对于复制因子为r的tcluster集群生成的聚合吞吐量、代理存储收到的吞吐量如下：

```
t[storage] = t[cluster]/#brokers + t[cluster]/#brokers * (r-1)
           = t[cluster]/#brokers * r
```

这一点可以进一步简化：

```
max(t[cluster]) <= max(t[storage]) * #brokers/r
```

使用此公式、您可以根据Kafka热层需求选择合适的ONTAP 平台。

下表说明了A900的预期生产者吞吐量以及不同的复制因素：

复制因子	生产者吞吐量**(GPP)**
3 (测量值)	3.4
2.	5.1
1.	10.2

结论

针对错误重命名问题的NetApp解决方案 为以前与NFS不兼容的工作负载提供了一种简单、廉价且集中管理的存储形式。

这种新模式使客户能够创建更易于管理的Kafka集群、这些集群更易于迁移和镜像、以实现灾难恢复和数据保护。

我们还发现、NFS还可以通过NetApp ONTAP 提供更多优势、例如降低CPU利用率和加快恢复速度、显著提高存储效率以及提高性能。

从何处查找追加信息

要了解有关本文档中所述信息的更多信息，请查看以下文档和 / 或网站：

- 什么是Apache Kafka?

  "https://www.confluent.io/what-is-apache-kafka/"

- 什么是愚蠢的重命名?

  "https://linux-nfs.org/wiki/index.php/Server-side_silly_rename"

- 正在为流式应用程序读取ONATP。

  "https://www.netapp.com/blog/ontap-ready-for-streaming-applications/"

- 愚蠢—将问题描述 重命名为Kafka。

  "https://sbg.technology/2018/07/10/kafka-nfs/"

- NetApp 产品文档

  "https://www.netapp.com/support-and-training/documentation/"

- 什么是NFS?

  "https://en.wikipedia.org/wiki/Network_File_System"

- 什么是Kafka分区重新分配?

  "https://docs.cloudera.com/runtime/7.2.10/kafka-managing/topics/kafka-manage-cli-reassign-overview.html"

- 什么是Open消息 基准?

  "https://openmessaging.cloud/"

- 如何迁移Kafka代理?

  "https://medium.com/@sanchitbansal26/how-to-migrate-kafka-cluster-with-no-downtime-58c216129058"

- 您如何监控与Prometheus的Kafka代理?

  https://www.confluent.io/blog/monitor-kafka-clusters-with-prometheus-grafana-and-confluent/

- 适用于Apache Kafka的托管平台

  https://www.instaclustr.com/platform/managed-apache-kafka/

- 支持Apache Kafka

https://www.instaclustr.com/support-solutions/kafka-support/

- Apache Kafka的咨询服务

  https://www.instaclustr.com/services/consulting/

版权信息

版权所有 © 2024 NetApp, Inc.。保留所有权利。中国印刷。未经版权所有者事先书面许可，本文档中受版权保护的任何部分不得以任何形式或通过任何手段（图片、电子或机械方式，包括影印、录音、录像或存储在电子检索系统中）进行复制。

从受版权保护的 NetApp 资料派生的软件受以下许可和免责声明的约束：

本软件由 NetApp 按“原样”提供，不含任何明示或暗示担保，包括但不限于适销性以及针对特定用途的适用性的隐含担保，特此声明不承担任何责任。在任何情况下，对于因使用本软件而以任何方式造成的任何直接性、间接性、偶然性、特殊性、惩罚性或后果性损失（包括但不限于购买替代商品或服务；使用、数据或利润方面的损失；或者业务中断），无论原因如何以及基于何种责任理论，无论出于合同、严格责任或侵权行为（包括疏忽或其他行为），NetApp 均不承担责任，即使已被告知存在上述损失的可能性。

NetApp 保留在不另行通知的情况下随时对本文档所述的任何产品进行更改的权利。除非 NetApp 以书面形式明确同意，否则 NetApp 不承担因使用本文档所述产品而产生的任何责任或义务。使用或购买本产品不表示获得 NetApp 的任何专利权、商标权或任何其他知识产权许可。

本手册中描述的产品可能受一项或多项美国专利、外国专利或正在申请的专利的保护。

有限权利说明：政府使用、复制或公开本文档受 DFARS 252.227-7013（2014 年 2 月）和 FAR 52.227-19（2007 年 12 月）中“技术数据权利 — 非商用”条款第 (b)(3) 条规定的限制条件的约束。

本文档中所含数据与商业产品和/或商业服务（定义见 FAR 2.101）相关，属于 NetApp, Inc. 的专有信息。根据本协议提供的所有 NetApp 技术数据和计算机软件具有商业性质，并完全由私人出资开发。 美国政府对这些数据的使用权具有非排他性、全球性、受限且不可撤销的许可，该许可既不可转让，也不可再许可，但仅限在与交付数据所依据的美国政府合同有关且受合同支持的情况下使用。除本文档规定的情形外，未经 NetApp, Inc. 事先书面批准，不得使用、披露、复制、修改、操作或显示这些数据。美国政府对国防部的授权仅限于 DFARS 的第 252.227-7015(b)（2014 年 2 月）条款中明确的权利。

商标信息

NetApp、NetApp 标识和 http://www.netapp.com/TM 上所列的商标是 NetApp, Inc. 的商标。其他公司和产品名称可能是其各自所有者的商标。